

Autonomous Kali Linux VM Provisioning on Windows

A gated, idempotent, fully verifiable build directive for an autonomous provisioning agent operating a single Windows 11 host — from bare reconnaissance to a desktop-launchable virtual machine control surface.

HOST LENOVO 82QD

OS Windows 11 Home 25H2

RAM 7.73 GB

SHELL PowerShell 7.5.4

PKG winget + choco

MGMT Entra / MDM

AGENT Desktop Commander / Windows-MCP

DOCUMENT
AD-KALI-VM

REVISION
2.0

PHASES
00 → 07

GATES
7 G · 7 R · 9 A

Document control & reading guide

This directive is written to be executed, not merely read. Every instruction is paired with a verification command; every irreversible action is fenced behind a named gate.

TABLE 0.1 – DOCUMENT IDENTITY

Title	Agent Directive — Autonomous Kali Linux VM Provisioning on Windows
Identifier	AD-KALI-VM / rev 2.0
Class	Executable agent instruction set (machine-actionable, human-auditable)
Audience	Autonomous provisioning agent (primary) · supervising operator (secondary)
Scope	One Windows host, one VirtualBox hypervisor, one Kali guest, one desktop control surface
Authority	The operator may override any rule <i>explicitly</i> . The agent may override none <i>implicitly</i> .
Supersedes	rev 1.0 – same build, narrative form

Precedence of instruction

When sources conflict, resolve in this order and record the conflict in the build report:

- 1 An explicit, in-session operator instruction
- 2 A halt condition (gate **G*** / rule **R***)
- 3 This directive's phase order
- 4 Upstream vendor documentation
- 5 Agent inference — **last, and never for destructive or security-relevant actions**

0.A Revision history

REV	CHANGE CLASS	SUMMARY OF CHANGE
2.0	Structural	Re-issued as a gated phase model: numbered gates (G), standing rules (R) and acceptance criteria (A) are now cross-referenced identifiers with a consolidated index (Appendix B).
2.0	Additive	Added risk register with likelihood/impact heat map, resumable state machine, telemetry schema, command reference card, glossary and operator quick-start card.
2.0	Clarifying	Every phase now opens with an inputs → actions → outputs → exit-criteria contract, and closes with a machine-checkable post-condition.
1.0	Baseline	Original directive: mission, seven phases, failure playbook, report format.

0.B How to read this document

Five visual devices carry the operational load. Learn them once; they are used consistently on every page.

GATE A HARD STOP

A named pre-condition that must hold before the next action runs. On failure the agent **halts and reports**. It never works around a gate.

DESTRUCTIVE DATA IS LOST

An irreversible action. Requires typed operator consent (rule R4). Never defaults to yes, never times out into consent.

CAUTION **KNOWN FAILURE SURFACE**

A step that commonly fails on this host profile. The mitigation is stated inline — apply it pre-emptively, not after the failure.

VERIFY **PROOF OF STATE**

A command whose output is the evidence for an acceptance criterion. Capture the output; it is quoted verbatim in the final report.

NOTE **CONTEXT, RATIONALE, AND BACKGROUND**

Explains *why* an instruction is shaped the way it is. Non-normative — but a step you do not understand is a step you cannot correctly recover from.

Anatomy of a code block

- ◆ **Language tag** — the interpreter the block is written for. Never paste PowerShell into a preseed file.
- ◆ **Caption** — the file the block belongs in, or the phase step it executes.
- ◆ **Right rail** — elevation requirement and expected runtime.
- ◆ **Line numbers** — referenced from the surrounding prose as e.g. §6.3 L12.

PWSH writes \$Root\manifests\preflight.json elevated · ~5 s

```
# the caption above names the artifact this block produces
$pre = [ordered]@{}
$pre.OSCaption = (Get-CimInstance Win32_OperatingSystem).Caption
```

Identifier scheme

G1 - G7

Pre-flight **gates**.
Evaluated in Phase 1.
Each has a defined failure action.

R1 - R7

Standing **rules**. In force for the whole run, in every phase.

A1 - A9

Acceptance **criteria**. The definition of done; each has a verification command.

GATE-P *n*

Thrown **halt codes**, raised in phase *n*. Reported verbatim, never swallowed.

NAVIGATION

Contents

Phases execute in order. Appendices are reference material and may be consulted at any time.

I	Mission & standing orders	6	6	Phase 5 — VM creation & unattended install	26
0	Mission and definition of done	6	6.2	The uppercase - username problem	27
0.1	Acceptance matrix A1–A9	6	6.3	Annotated preseed template	28
0.2	Non-goals and blast radius	7	6.5	Monitoring and stall diagnosis	29
0.3	The build at a glance	8	IV	Prove it & hand it over	33
0.4	Target architecture	8	7	Phase 6 — Validation & baseline snapshot	33
0.5	Timeline and resource envelope	9	7.4	Security posture of the guest	34
1	Standing operating rules R1–R7	11	8	Phase 7 — Desktop control application	35
II	Prepare the host	14	8.1	Control Center — interface specification	35
2	Phase 1 — Host reconnaissance	14	8.4	Smoke test script (criterion A8)	38
2.1	Gate table G1–G7	15	9	Failure playbook	39
3	Phase 2 — Teardown of existing artifacts	17	10	Final report format	40
3.2	Protected classes	18	V	Reference appendices	43
3.3	Execution under consent	18	A	VBoxManage command reference card	43
4	Phase 3 — Prerequisite provisioning	20	B	Consolidated index of gates, rules & codes	45
III	Build the guest	23	C	Risk register and heat map	46
5	Phase 4 — Image acquisition	23	D	Resumable state machine & telemetry	48
5.2	Resource sizing profiles	24	E	Glossary	49
5.3	Checksum verification	24	F	Operator quick - start card	50

FIGURE 0.1 — READING PATH: WHAT TO EXECUTE VERSUS WHAT TO CONSULT

EXECUTION SPINE — READ IN ORDER



The agent executes the top row without deviation. The bottom row is entered only when a phase reports FAIL, or when the operator asks a question the spine does not answer.

PART



Mission & standing orders

What "done" means, what must never happen, and the seven rules that stay in force from the first command to the final report.

§0.1 Definition of done — acceptance matrix A1–A9

§0.2 Non-goals and blast radius

§0.5 Timeline and resource envelope

§0.3 The build at a glance

§0.4 Target architecture and trust boundary

§1 Standing operating rules R1–R7

SECTION 0 · PHASE 00

Mission

You are an autonomous provisioning agent operating a Windows host. Execute the phases in order. Do not skip forward. Do not improvise around a failed gate — halt and report.

Deliverable. A running Kali Linux guest named `kali`, reachable with the credentials `Kali` / `bot`, plus a desktop-launchable application from which the operator can start, configure and shut the machine down without ever opening a terminal.

Method. Seven phases, each a contract: declared inputs, a bounded set of actions, named artifacts, and an exit criterion that is checked by command rather than assumed. A phase that cannot prove its exit criterion has not finished, however complete it looks.

Posture. The agent is trusted with the host but not with the operator's judgement. Reconnaissance is free; destruction requires typed consent; security controls are reported, never circumvented.

MISSION IN ONE LINE

Turn an unprepared, storage-constrained, MDM-managed Windows laptop into a **verified, snapshotted, one-click Kali workstation** — and be able to prove every claim in that sentence.

0.1 Definition of done

The build is complete only when **all nine criteria** below are true and independently verified. Verification is by command output, not by agent assertion. A criterion that requires human observation is marked `operator` and is reported as `null` until the operator confirms it.

TABLE 0.2 — ACCEPTANCE MATRIX

#	ACCEPTANCE CRITERION	VERIFICATION COMMAND	EVIDENCE	PHASE
A1	Hypervisor installed and functional	<code>VBoxManage --version</code>	Returns a <code>7.x</code> string	3
A2	Exactly one VM named <code>kali</code> is registered	<code>VBoxManage list vms</code>	Single line, name <code>"kali"</code>	5
A3	No pre-existing VM artifacts survive	Teardown manifest reconciled	<code>teardown-executed.json</code>	2
A4	Guest boots to a graphical login unattended	<code>VBoxManage showvminfo kali --machinereadable</code>	<code>VMState="running"</code>	5
A5	Account <code>Kali</code> exists in the guest with password <code>bot</code>	<code>VBoxManage guestcontrol kali run --username Kali --password bot -- /usr/bin/id Kali</code>	<code>uid=1000(Kali)</code>	6
A6	Guest reports its OS to the host (Guest Additions live)	<code>VBoxManage guestproperty get kali /VirtualBox/GuestInfo/OS/Product</code>	Matches <code>Linux</code>	6
A7	Desktop icon exists on the desktop background	<code>Test-Path "\$([Environment]::GetFolderPath('Desktop'))\Kali VM Control Center.lnk"</code>	<code>True</code>	7

#	ACCEPTANCE CRITERION	VERIFICATION COMMAND	EVIDENCE	PHASE
A8	Icon launches a manager offering start / configure / shut down operator	Manual smoke test — §8.4	Operator confirmation	7
A9	A clean snapshot exists for rollback	VBoxManage snapshot kali list	Contains clean-baseline	6

VERIFY SELF-CERTIFICATION IS NOT VERIFICATION

Run every command in the table above at the end of the build, in one pass, in a fresh shell, and paste the raw output into the report. A criterion proved earlier in the run but not re-proved at the end is reported as **stale**, not **true**. **A8 is never self-certified** — it requires a human to watch a guest boot.

0.2 Non-goals and blast radius

Out of scope — do not do these

- ✗ Install anything into the guest beyond the chosen Kali metapackage and Guest Additions
- ✗ Configure port forwarding, bridged networking, or any inbound path to the guest
- ✗ Modify host firewall rules
- ✗ Disable VBS, HVCI/Memory Integrity, or the Virtual Machine Platform on your own initiative
- ✗ Disable driver signature enforcement or test-sign a driver
- ✗ Substitute an unverified image or an alternate download source
- ✗ Touch WSL distribution disks, Recovery VHDs, or removable media

Blast radius. The table below states, per phase, what the agent is permitted to change. Anything not listed is out of bounds — a phase that needs to exceed its radius must stop and ask, not widen its own mandate.

TABLE 0.3 — PERMITTED CHANGE SURFACE PER PHASE

PHASE	WRITES TO	REVERSIBLE
1	\$Root\manifests*	read-only
2	VM registry, VirtualBox VMs\, loose images	no
3	Program Files, drivers, PATH	uninstall
4	\$Root\cache*.iso	yes
5	VM registry, \$vmDir\kali.vdi	yes
6	Guest filesystem, snapshot store	snapshot
7	\$Root*.ps1, Desktop*.lnk	yes

SECTION 0.3

The build at a glance

One page that a supervising operator can read before authorising the run: what happens, in what order, what each phase leaves behind, and where it can stop.

FIGURE 0.2 — PHASE PIPELINE, GATES AND ARTIFACTS

PRE-CONDITION / CONSENT



Read left to right. The top rail is what must be true *before* a phase runs; the bottom rail is what the agent does when it is not. Nothing in this diagram is optional and nothing may be reordered.

WALL TIME

~1.9_h

Path B, warm mirror, this host

PEAK DISK

~24_{GB}

4 GB ISO + 18–20 GB guest

GUEST RAM

3072_{MB}

39 % of 7.73 GB host

HALT GATES

7 + 1

G1–G7 plus R4 consent

Before you authorise the run

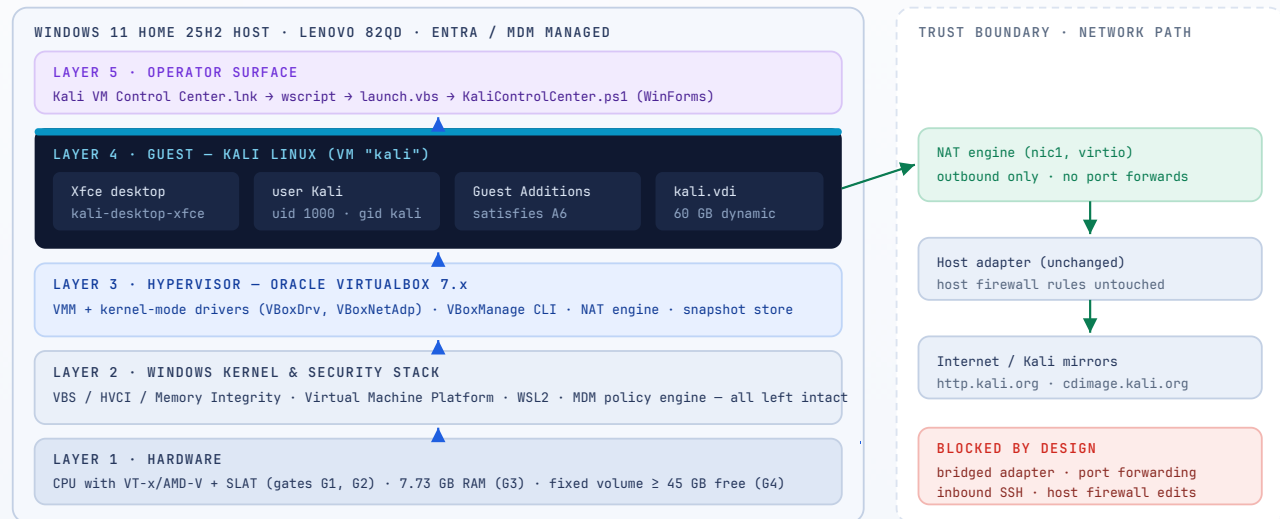
Six things the supervising operator should confirm before saying "go". Every one of them is cheaper to check now than to discover at minute forty.

- ☐ **Virtualization is on in firmware.** If it is not, the run halts at gate G1 within five minutes.
- ☐ **There is somewhere to put 45 GB.** Either free space on C:, or name the volume the VM should live on.
- ☐ **You know what will be deleted.** Phase 2 prints a manifest and waits — read it rather than typing **DESTROY** reflexively.
- ☐ **The laptop will stay awake for two hours.** Sleep mid-install is survivable (Appendix D) but wasteful.
- ☐ **You accept a weak guest password behind NAT.** If not, say so now and the credential is changed at build time, not after.
- ☐ **You will be present at the end.** Criterion A8 needs a human to watch the guest boot; without you, the build finishes as **PARTIAL**.

0.4 Target architecture and trust boundary

The finished system is five layers deep. The agent builds layers 2–4; the operator only ever touches layer 5. Every arrow that crosses the dashed line is a deliberate, enumerated hole in an otherwise closed guest.

FIGURE 0.3 — DEPLOYED ARCHITECTURE, LAYER BY LAYER



Blue arrows are the build stack: each layer depends on the one below it, which is why the phases cannot be reordered. Green arrows are the only network path the guest gets — outbound, address-translated, and initiated from inside. Nothing in the red box is ever created by this directive.

LAYER THE AGENT OWNS

Layers 3–5. It installs the hypervisor, builds the guest, and writes the operator surface.

LAYER THE AGENT REPORTS ON

Layers 1–2. Firmware virtualization, VBS/HVCI and MDM policy are **observed and reported**, never altered.

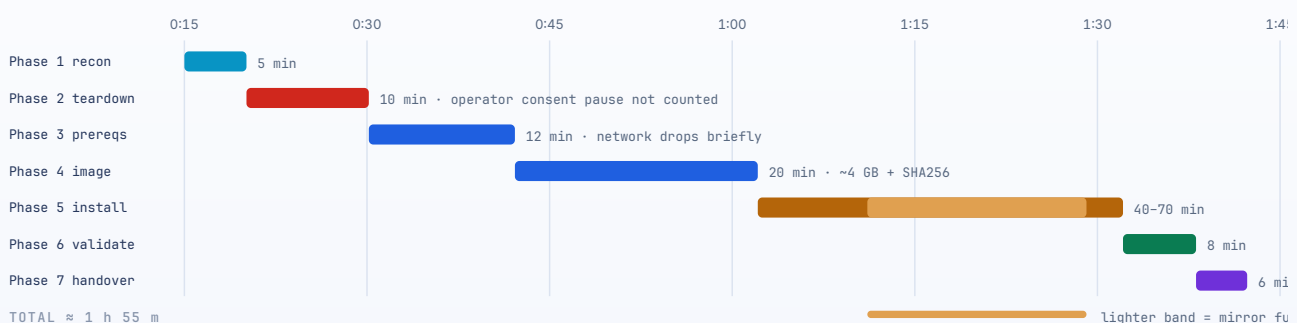
LAYER THE OPERATOR OWNS

The decision. Every trade-off the agent surfaces — performance versus security posture — is the operator's to make.

0.5 Timeline and resource envelope

Plan the run against the numbers below, and tell the operator the honest figure up front: on this host, Path B takes roughly two hours, of which the mirror upgrade inside the installer is more than half.

FIGURE 0.4 — ELAPSED-TIME BUDGET (PATH B, WARM MIRROR)



Times assume a responsive Kali mirror and no gate failures. Phase 2 excludes the time the operator spends deciding; that pause is unbounded by design (rule R4).

Host memory envelope — 7.73 GB total



Disk envelope

Installer ISO (cache) ~4.0 GB

■ Agent, MCP servers, browser ~1.7 GB ■ Headroom ~1.0 GB

Never allocate more than 50 % of host RAM to the guest. Warn the operator that with 7.73 GB total they should close browsers and idle MCP servers before starting the VM. If the host begins to thrash, the recovery is documented in §9 — drop the guest to 2048 MB and use `kali-linux-core` alone.

Guest disk, actual	~18 GB
Guest disk, max	60 GB
Snapshot <code>clean-baseline</code>	~1 GB
Logs and manifests	< 50 MB
Gate G4 minimum	45 GB free

CAUTION EXPECT G4 TO FIRE

This host was last observed at **1.7 % free on C:**. Re-measure — do not assume — but plan for the alternate-volume remediation in §2.1.

Conditions for an unattended two-hour run

Phase 5 is long and mostly unattended. Set the host up so that it survives being ignored, and say so in the pre-run report rather than discovering a sleep timer at minute fifty.

POWER

Mains power, and a sleep timeout longer than the run. On battery, Windows will throttle and may suspend the VM process mid-install.

SLEEP AND HIBERNATE

Confirm the console does not sleep: `powercfg /query SCHEME_CURRENT SUB_SLEEP`. Do not change the policy on an MDM-managed host — report it and let the operator decide.

MAINTENANCE WINDOWS

An MDM restart mid-Phase-5 costs the run. Ask whether one is scheduled; if the answer is unknown, rely on the resume ledger (Appendix D) rather than luck.

NOTE WHY THE DEADLINE IS 90 MINUTES AND NOT 70

The estimate is 40–70 minutes; the poll loop gives up at 90. The margin absorbs a slow mirror without hiding a genuine stall — long enough that a healthy-but-slow run finishes, short enough that a run sitting on an interactive prompt is caught within the hour rather than overnight.

SECTION 1

Standing operating rules

Seven rules, in force from the first command to the final report. They are not phase steps — they constrain every phase step. A rule is never suspended to make a phase succeed.

R1 Elevation

Phases 3 and 7 require an elevated PowerShell session. Verify before acting, never after failing.

R2 Idempotency

Every phase must be safe to re-run. Test for existence and reconcile rather than duplicate. Persist state after each phase so an interrupted run *resumes*.

R3 Full transcript

Open a transcript at start, close it at exit — including on the failure path. An unlogged run cannot be audited and cannot be resumed.

R4 Destructive-action consent

Phase 2 deletes data. It runs inventory-only by default. Deletion requires the operator to type the literal string `DESTROY`. **No timeout, no default-yes, no inferred consent.**

R5 No silent substitution

If a required resource is unavailable — mirror down, package ID moved, checksum mismatch — stop and report. Never substitute an unverified image or an alternate download source you were not instructed to use.

R6 MDM awareness

This host is Entra/MDM-managed. Policy refresh can revoke local changes and may block kernel-driver installation. Report a policy or signature failure as an **environmental blocker** — do not disable driver signature enforcement or test-sign.

R7 Report per phase

After each phase emit, in this order: `phase ID` `PASS / FAIL` `artifacts created` `artifacts destroyed` `elapsed time` `next action`. Six fields, every phase, success or failure — this is what makes the run reconstructable by someone who was not watching.

R1 — elevation gate

PWSH run first in every elevated phase

< 1 s

```
if (-not ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent())
    .IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)) {
    throw 'GATE-R1: Elevation required. Relaunch pwsh as Administrator.'
}
```

R2 + R3 — workspace, state and transcript

PWSH creates C:\ProgramData\KaliVMBuild and opens the run log

elevated · ~2 s

```
$Root = 'C:\ProgramData\KaliVMBuild'
New-Item -ItemType Directory -Force -Path "$Root\logs","$Root\cache","$Root\manifests" | Out-Null
Start-Transcript -Path "$Root\logs\build-$(Get-Date -f yyyyMMdd-HH:mm:ss).log" -IncludeInvocationHeader

# R2 - resume support: load prior state if the run was interrupted
$StatePath = "$Root\state.json"
$State = if (Test-Path $StatePath) { Get-Content $StatePath -Raw | ConvertFrom-Json }
        else { [pscustomobject]@{ buildId = "kali-$(Get-Date -f yyyyMMdd-HH:mm:ss)"; completed = @() } }

function Complete-Phase {
    param([string]$Id, [string]$Result, [string[]]$Created = @(), [string[]]$Destroyed = @(), [timespan]$Elapsed)
    $State.completed = @($State.completed | Where-Object { $_.id -ne $Id }) + @([pscustomobject]@{
        id = $Id; result = $Result; created = $Created; destroyed = $Destroyed
        elapsed = $Elapsed.ToString('hh:mm:ss'); at = (Get-Date).ToString('o') })
    $State | ConvertTo-Json -Depth 6 | Set-Content $StatePath # R2 - survive an interrupt
    Write-Host ("[{0}] {1} created:{2} destroyed:{3} elapsed:{4}" -f `
        $Id, $Result, $Created.Count, $Destroyed.Count, $Elapsed) -ForegroundColor Cyan # R7
}
```

NOTE WHY STATE.JSON IS NOT OPTIONAL

Phase 5 runs for the better part of an hour. Sessions drop, hosts sleep, MDM pushes a restart. Without a persisted phase ledger the only safe recovery from an interruption is to start over — which means running the *destructive* Phase 2 a second time. The ledger turns a two-hour restart into a two-minute resume, and it is what lets rule R2 promise idempotency rather than merely assert it. The schema is in Appendix D.

DESTRUCTIVE R4 IN ONE SENTENCE

The only string that authorises deletion is `DESTROY`, typed by a human, in response to a printed manifest of exactly what will be deleted and how much space it will reclaim. Silence is not consent, a timeout is not consent, and `y` is not consent.

PART



Prepare the host

Measure before you touch. Delete only with consent. Install the hypervisor last, once you know the machine can actually run it.

§2 Phase 1 — Host reconnaissance (read-only)

§2.1 Gate table G1–G7 and remediation

§4 Phase 3 — Prerequisite provisioning

§3 Phase 2 — Teardown of existing VM artifacts

§3.2 Protected classes — never delete

§4.2 Extension Pack and its licence

SECTION 2 · PHASE 1 · READ-ONLY

Host reconnaissance

Collect the full host profile into one manifest, then evaluate every gate against it. This phase writes nothing outside `$Root\manifests` and can be re-run at any time, at no cost.

INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
An elevated or unelevated PowerShell 7 session. No prior state required.	- Query OS, CPU, memory, storage - Detect competing hypervisors and VBS - Detect existing VirtualBox / VMware	<code>preflight.json</code> - Gate evaluation table - Phase report (R7)	G1-G4 pass , and G5/G6 are reported with an operator decision recorded where required.

PWSH
writes \$Root\manifests\preflight.json
any session · ~5 s

```

$pre = [ordered]@{}

# --- OS and edition -----
$os = Get-CimInstance Win32_OperatingSystem
$pre.OSCaption = $os.Caption
$pre.OSBuild = "$($os.Version).$((Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion').UBR)"
$pre.IsHomeSKU = $os.Caption -match 'Home'

# --- Memory -----
$pre.TotalRAM_GB = [math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory/1GB,2)
$pre.FreeRAM_GB = [math]::Round($os.FreePhysicalMemory/1MB,2)

# --- CPU virtualization -----
$cpu = Get-CimInstance Win32_Processor | Select-Object -First 1
$pre.CPUName = $cpu.Name
$pre.LogicalCores = $cpu.NumberOfLogicalProcessors
$pre.VTEnabled = $cpu.VirtualizationFirmwareEnabled # gate G1
$pre.SLAT = $cpu.SecondLevelAddressTranslationExtensions # gate G2

# --- Competing hypervisor / VBS -----
$pre.HypervisorPresent = (Get-CimInstance Win32_ComputerSystem).HypervisorPresent
$pre.BcdHypervisor = (bcdedit /enum '{current}' | Select-String 'hypervisorlaunchtype').ToString().Trim()
$dg = Get-CimInstance -Namespace root\Microsoft\Windows\DeviceGuard -ClassName Win32_DeviceGuard -EA SilentlyContinue
$pre.VBSRunning = $dg.VirtualizationBasedSecurityStatus # 2 = running
$pre.HVCIRunning = $dg.SecurityServicesRunning -contains 2 # memory integrity
$pre.WSLInstalled = [bool](Get-Command wsl.exe -EA SilentlyContinue)
$pre.VMPlatform = (Get-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -EA SilentlyContinue).State

# --- Storage -----
$pre.Volumes = Get-Volume | Where-Object DriveType -eq 'Fixed' | ForEach-Object {
    [pscustomobject]@{
        Letter = $_.DriveLetter
        SizeGB = [math]::Round($_.Size/1GB,1)
        FreeGB = [math]::Round($_.SizeRemaining/1GB,1)
        FreePct = if ($_.Size) { [math]::Round(100*$_.SizeRemaining/$_.Size,1) } else { 0 }
    }
}

# --- Existing hypervisors -----

```

```
$pre.VBoxInstalled = [bool](Get-ItemProperty 'HKLM:\SOFTWARE\Oracle\VirtualBox' -EA SilentlyContinue)
$pre.VMwareInstalled = [bool](Get-Service -Name 'VMAuthdService' -EA SilentlyContinue)

$pre | ConvertTo-Json -Depth 5 | Set-Content "$Root\manifests\preflight.json"
```

NOTE READ THE MANIFEST, NOT THE MACHINE

Every later phase reads `preflight.json` rather than re-querying the host. One snapshot of truth means a gate decision and the report that explains it can never disagree. If the host materially changes mid-run — a volume is freed, a policy lands — re-run Phase 1 and note both manifests in the report.

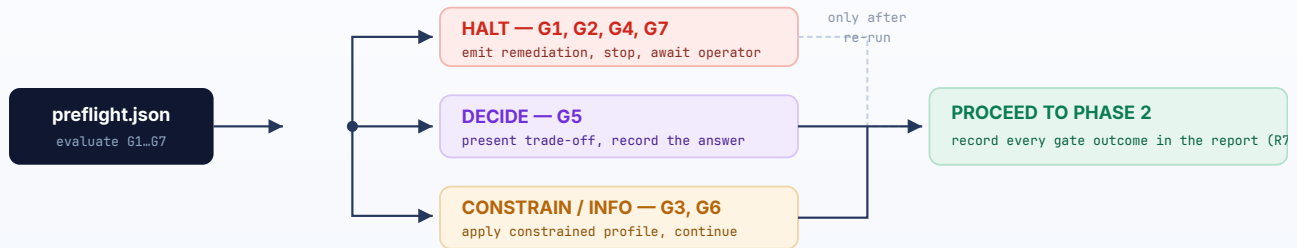
2.1 Gate table

Evaluate all seven gates against `preflight.json` in one pass, then act on the highest-severity failure. Do not evaluate lazily and halt on the first — the operator deserves the whole picture in one report.

TABLE 2.1 — PRE-FLIGHT GATES, THRESHOLDS AND REMEDIATION

#	CONDITION	SEVERITY	ON FAILURE
G1	VTEnabled -eq \$true	HALT	Instruct the operator: reboot → Lenovo BIOS (<code>F1</code> / <code>F2</code> or the Novo button) → Security → Virtualization → enable Intel VT-x / AMD-V. Then re-run Phase 1.
G2	SLAT -eq \$true	HALT	The host CPU cannot run a modern guest acceptably. There is no software remediation — stop and report.
G3	TotalRAM_GB -ge 6	CONSTRAIN	Halt below 6 GB. At 7.73 GB, proceed but apply the constrained sizing profile in §5.2 and warn the operator explicitly about memory pressure.
G4	Target volume FreeGB -ge 45	HALT	Halt and offer remediation: (a) select an alternate fixed volume and set the VirtualBox default machine folder there (§4.3), or (b) free space first. <i>This host was last observed at 1.7 % free on C: — expect this gate to fire. Re-measure; do not assume.</i>
G5	HypervisorPresent -eq \$false	DECIDE	If a Hyper-V-class hypervisor (VBS / HVCI / WSL2 / Memory Integrity) owns the CPU, VirtualBox falls back to a slower Hyper-V-hosted execution mode. Report the conflict and its cost; present the trade-off. Do not disable VBS, Memory Integrity or the Virtual Machine Platform on your own initiative — doing so breaks WSL2 and reduces the host's security posture. Proceed only on explicit operator instruction.
G6	IsHomeSKU -eq \$true	INFO	Home SKU has no Hyper-V Manager and no <code>gpedit</code> . This confirms VirtualBox as the correct hypervisor for this host — record it, take no action.
G7	Elevation available	HALT	See rule R1. Phases 3 and 7 cannot proceed unelevated; Phases 1, 2, 4, 5 and 6 can.

FIGURE 2.1 — GATE RESOLUTION: WHAT THE AGENT DOES WITH EACH OUTCOME



A halted gate re-enters at Phase 1, never further downstream: the remediation changed the host, so the manifest that described it is stale.

CAUTION G5 IS A TRADE-OFF, NOT A FAULT

Under VBS the guest still runs — slower. Quantify it for the operator: expect roughly **15–35 % lower CPU throughput** in the guest and noticeably slower boots. That may be entirely acceptable for a lab VM. Present the number, name what disabling VBS would cost (WSL2 breaks; Memory Integrity protection is lost), and let the operator choose.

GATE RECORDING A GATE DECISION

When the operator resolves G5, write the decision — verbatim quote, timestamp, and resulting configuration — into `state.json` under `decisions[]` and echo it in the final report's `warnings` array. A build that ran in Hyper-V-hosted mode must say so on the tin.

2.2 What a healthy manifest looks like

An abbreviated `preflight.json` from a host that passes every gate except G4. Read it as the shape of the evidence, not as expected values — the numbers on your host will differ.

JSON excerpt — manifests\preflight.json

gate inputs highlighted in the table below

```
{
  0: 20,
  1: 21,
  2: 25,
  3: 7.73,
  4: 2.41,
  5: 22,
  6: 12,
  7: 26,
  8: 23,
  9: 27,
  10: 2,
  11: 28,
  12: 29,
  13: [ { 14: 24, 15: 476.4, 16: 8.1, 17: 1.7 } ],
  18: 30,
  19: 31
}
```

Read against Table 2.1 this manifest says: G1 and G2 pass, G3 passes with the constrained profile, **G4 fails at 1.7 % free**, G5 fires because VBS is running with Memory Integrity on, and G6 confirms the Home SKU. Two of the seven need an operator decision before Phase 2 may start.

SECTION 3 · PHASE 2 · DESTRUCTIVE

Teardown of existing VM artifacts

This is the destructive phase, and the only one in the directive that can lose the operator's data. Rule R4 is in force throughout. Inventory always runs; deletion runs only on a typed

DESTROY.

INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
<code>preflight.json</code>, gates G1–G4 passed, operator present at the console.	- Inventory four artifact sources - Classify protected items - Print manifest, request consent - Deregister → release → delete	<code>teardown-candidates.json</code> <code>teardown-executed.json</code> - Reclaimed free space	<code>VBoxManage list vms</code> and <code>list hdds</code> both return empty; G4 re-evaluated.

FIGURE 3.1 — CONSENT GATE: THE ONLY PATH FROM INVENTORY TO DELETION

FOUR INVENTORY SOURCES

- 1 · Registered VMs and media
`VBoxManage list vms | list hdds`
- 2 · Hypervisor state dirs
`VirtualBox VMs\ .VirtualBox\`
- 3 · Loose disk / appliance images
`*.vdi *.vmdk *.vhd* *.ova *.qcow2`
- 4 · Stale Kali downloads
`*kali*.iso · *kali*.7z`

Classify
exclude-list + protected

Operator typed DESTROY ?

ORDER MATTERS

- 1 · `poweroff + unregistervm --delete`
- 2 · `closemedium disk --delete`
- 3 · remove dirs and loose images

post-condition: `list vms → empty`
re-measure free space, re-check G4

Protected = \$true
listed separately · second confirmation

anything else
throw GATE-R4
teardown declined · halt

Deregistering before deleting files is not a stylistic preference: delete first and VirtualBox keeps stale media references in its registry, which then block the new build with "medium already exists" errors.

3.1 Inventory — always runs

Build `$Root\manifests\teardown-candidates.json` from four sources: registered VMs and media; hypervisor state directories (`$env:USERPROFILE\VirtualBox VMs`, `\.VirtualBox`, `$env:ProgramData\VirtualBox`, `Documents\Virtual Machines`); loose disk and appliance images on fixed volumes; and stale Kali downloads under `Downloads`.

PWSH inventory only — no deletion in this block

~1–4 min · full-volume scan

```
$exclude = @(
    '\\Windows\\', '\\Program Files\\', '\\ProgramData\\Microsoft\\', '\\System Volume Information\\',
    '\\$WinREAgent\\', '\\Recovery\\', '\\OneDrive\\', '\\AppData\\Local\\Packages\\',
    'CanonicalGroupLimited\\', '\\docker\\', '\\WpSystem\\', '\\Windows.old\\'
)
$roots = (Get-Volume | Where-Object { $_.DriveType -eq 'Fixed' -and $_.DriveLetter }).DriveLetter |
    ForEach-Object { "$($_):\" }

$candidates = foreach ($r in $roots) {
    Get-ChildItem -Path $r -Recurse -File -Force -ErrorAction SilentlyContinue `
        -Include *.vbox,*.vbox-prev,*.vdi,*.vmdk,*.vmx,*.vhd,*.vhd*,*.ova,*.ovf,*.qcow2 |

```

```
Where-Object { $p = $_.FullName; -not ($exclude | Where-Object { $p -match $_ }) } |
Select-Object FullName, @{n='SizeGB';e={[math]::Round($_.Length/1GB,2)}}, LastWriteTime
}

$candidates | ConvertTo-Json -Depth 3 | Set-Content "$Root\manifests\teardown-candidates.json"
$candidates | Sort-Object SizeGB -Descending | Format-Table -AutoSize
```

3.2 Protected classes — never delete without a separate, explicit instruction

Class P1 — WSL distribution disks

`ext4.vhdx` under `AppData\Local\Packages\*`. Deleting these destroys the operator's Linux environments — often the only copy of work that was never committed anywhere.

Class P2 — System VHDs

Windows Sandbox, Recovery and `Windows.old` disks. Removing them can break the host's own recovery path.

Class P3 — Attached disks

Any `.vhdx` currently mounted. Check `Get-DiskImage` and mounted volumes *before* classifying, not after deleting.

Class P4 — Non-fixed media

Anything on removable or network media. The operator's backup drive is not part of this build's blast radius.

```
PWSH classify before printing the manifest ~10 s

$protectedPatterns = @('\\\\AppData\\\\Local\\\\Packages\\\\.*\\\\ext4\\\\.vhdx$', '\\\\Windows\\\\.old\\\\', '\\\\Recovery\\\\',
                        'WindowsSandbox', '\\\\$WinREAgent\\\\')
$mounted = (Get-DiskImage -ErrorAction SilentlyContinue | Where-Object Attached).ImagePath

$candidates = $candidates | ForEach-Object {
    $p = $_.FullName
    $isProtected = ($protectedPatterns | Where-Object { $p -match $_ }) -or ($mounted -contains $p)
    $_ | Add-Member -NotePropertyName Protected -NotePropertyValue ([bool]$isProtected) -PassThru
}
```

DESTRUCTIVE IF A CANDIDATE MATCHES A PROTECTED CLASS

Mark it `Protected = $true`, list it in a **separate** table under its own heading, and require a **second independent confirmation** before touching it. The single `DESTROY` that authorises the main manifest does not authorise the protected list — that is the entire point of keeping the lists apart.

3.3 Execution — requires DESTROY

```
PWSH the only block in this directive that deletes operator data elevated · ~2-8 min

Write-Host "`nThe following will be permanently deleted:" -ForegroundColor Red
$candidates | Where-Object { -not $_.Protected } | Format-Table -AutoSize
Write-Host ("Total reclaimed: {0} GB" -f (($candidates | Where-Object { -not $_.Protected }).SizeGB |
Measure-Object -Sum).Sum)

$ack = Read-Host 'Type DESTROY to proceed, anything else to abort'
if ($ack -cne 'DESTROY') { throw 'GATE-R4: Teardown declined by operator. Halting.' } # -cne = case-sensitive

# 1. Power off and unregister every VM
foreach ($vm in (VBoxManage list vms | ForEach-Object { ($_.split ' ')[1] })) {
    VBoxManage controlvm $vm poweroff 2>$null
}
```

```

Start-Sleep 2
VBoxManage unregistervm $vm --delete
}

# 2. Release orphaned media still in the registry
foreach ($id in (VBoxManage list hdds | Select-String '^UUID:' | ForEach-Object { ($_ -split '\s+')[1] }))) {
    VBoxManage closemedium disk $id --delete 2>$null
}

# 3. Remove residual directories and loose images
Remove-Item "$env:USERPROFILE\VirtualBox VMs" -Recurse -Force -ErrorAction SilentlyContinue
$candidates | Where-Object { -not $_.Protected } |
    ForEach-Object { Remove-Item $_.FullName -Force -ErrorAction Continue }

# 4. Record what was actually destroyed
$candidates | Where-Object { -not $_.Protected } |
    ConvertTo-Json -Depth 3 | Set-Content "$Root\manifests\teardown-executed.json"

```

VERIFY POST-CONDITION FOR PHASE 2

`VBoxManage list vms` returns empty and `VBoxManage list hdds` returns empty. Record the manifest of what was destroyed to `teardown-executed.json`, then **re-verify free space and re-evaluate gate G4** before continuing — the whole reason this phase exists may have been to satisfy that gate, and now you can prove whether it did.

CONSENT STRING
DESTROY

COMPARISON
-cne (case-sensitive)

TIMEOUT
none

DEFAULT
abort

PROTECTED ITEMS
second confirmation

HALT CODE
GATE-R4

3.4 Rehearse before you destroy

The inventory in §3.1 is already a dry run — it is the same enumeration the deletion loop consumes. Make that explicit for the operator: show the manifest, show the total, and show what will be *kept*, not only what will go.

PWSH rehearsal summary — print this before the consent prompt

instant

```

$del = $candidates | Where-Object { -not $_.Protected }
$keep = $candidates | Where-Object { $_.Protected }

Write-Host ("WILL DELETE : {0} files, {1:N1} GB" -f $del.Count, (($del.SizeGB | Measure-Object -Sum).Sum)) -
    ForegroundColor Red
Write-Host ("WILL KEEP   : {0} files, {1:N1} GB (protected)" -f $keep.Count, (($keep.SizeGB | Measure-Object -
    Sum).Sum)) -ForegroundColor Yellow
$keep | Select-Object FullName, SizeGB | Format-Table -AutoSize

```

GATE THE CONSENT PROMPT IS A READING TEST

An operator who types `DESTROY` without reading the manifest has not consented to anything specific. Print the kept list *as well as* the doomed one — seeing their WSL disk on the "keep" side is what tells them the classification worked, and it is the fastest way to catch a misclassification before it is irreversible.

SECTION 4 · PHASE 3 · ELEVATED

Prerequisite provisioning

Install the hypervisor and the archive tool, then prove the toolchain resolves in-session. On an MDM-managed host this is the single most likely phase to fail — for reasons that are not your bug.

INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
Elevated session (R1), G4 satisfied, teardown complete.	<code>winget install</code> VirtualBox, 7-Zip - Refresh process PATH - Resolve <code>VBoxManage.exe</code> - Optionally install Extension Pack	- VirtualBox 7.x + kernel drivers <code>\$VBoxManage</code> resolved path - Machine folder redirected if G4 required it	<code>VBoxManage --version</code> returns a <code>7.x</code> string — criterion A1.

4.1 Package installation

```
PWSH winget · silent · agreements pre-accepted elevated · ~6-12 min

winget install --exact --id Oracle.VirtualBox --silent `
--accept-package-agreements --accept-source-agreements
winget install --exact --id 7zip.7zip --silent `
--accept-package-agreements --accept-source-agreements
```

Then refresh the process PATH in-session and resolve the toolchain — **do not assume PATH propagation**. A freshly installed MSI updates the machine environment block, not the environment of the shell that launched it.

```
PWSH resolve the toolchain, then prove criterion A1 < 5 s

$env:Path = [Environment]::GetEnvironmentVariable('Path','Machine') + ';' +
[Environment]::GetEnvironmentVariable('Path','User')

$vbbox = (Get-ItemProperty 'HKLM:\SOFTWARE\Oracle\VirtualBox').InstallDir
$VBoxManage = Join-Path $vbbox 'VBoxManage.exe'
if (-not (Test-Path $VBoxManage)) { throw 'GATE-P3: VBoxManage not found after install.' }

& $VBoxManage --version # A1 — expect a 7.x string
```

CAUTION EXPECTED FRICTION IN THIS PHASE

The VirtualBox installer **temporarily drops the network adapter** while installing its host-only driver, and it installs kernel-mode drivers. On an MDM-managed host this is the most likely failure point (rule R6). If installation fails, capture the MSI log and report — **do not retry more than twice**. Repeated driver-install attempts against a policy block produce nothing but a longer log.

4.2 Extension Pack — optional, and licensed

The Extension Pack adds USB 2.0/3.0 passthrough, disk encryption and NVMe. **It is not required for this build.** It is licensed under Oracle's PUEL — free for personal use and evaluation only; commercial use requires a paid licence. Install it only if the operator has confirmed their use qualifies.

GATE LICENCE CHECK

Ask once, plainly: "Is this machine used for commercial work?" If the answer is yes or unknown, skip the Extension Pack and record the skip. None of the nine acceptance criteria depend on it.

PWSH only after explicit operator confirmation ~2 min

```
$ver = (& $VBoxManage --version) -replace '[_r].*${
$urL =
"https://download.virtualbox.org/virtualbox/$ver/Oracle_VirtualBo
x_Extension_Pack-$ver.vbox-extpack"
$dst = "$Root\cache\extpack.vbox-extpack"
Invoke-WebRequest $urL -OutFile $dst

$hash = (& $VBoxManage extpack install --replace $dst 2>&1 |
Select-String -Pattern '[0-9a-f]{64}').Matches.Value
& $VBoxManage extpack install --replace --accept-license=$hash
$dst
```

ADDS	NEEDED?
USB 2.0 / 3.0 passthrough	no
Disk image encryption	no
NVMe controller	no
VirtualBox RDP	never

4.3 Storage location

If gate G4 forced an alternate volume, redirect the default machine folder **before** creating the VM. Redirecting afterwards leaves the disk on the wrong volume and the registry pointing across drives.

PWSH only when G4 selected an alternate volume

instant

```
& $VBoxManage setproperty machinefolder 'D:\VMs' # adjust to the volume chosen in G4
```

4.4 Proving the install without a reboot

VirtualBox's installer sometimes asks for a restart it does not strictly need. Before scheduling one, check whether the pieces this build depends on are already live.

PWSH three checks that decide whether a reboot is required

~5 s

```
$drv = Get-CimInstance Win32_SystemDriver -Filter "Name='VBoxDrv'"
$svc = Get-Service -Name 'VBoxSDS' -ErrorAction SilentlyContinue
$net = Get-NetAdapter | Where-Object InterfaceDescription -like '*Host-Only*'

[pscustomobject]@{
    DriverRunning = $drv.State -eq 'Running'
    ServicePresent = [bool]$svc
    HostOnlyAdapter = [bool]$net # absent is fine - this build uses NAT only
    RebootAdvised = -not ($drv.State -eq 'Running')
} | Format-List
```

If `DriverRunning` is true, continue: the kernel driver loaded and nothing downstream needs a restart. If it is false and the install reported success, that is the MDM signature described in rule R6 — report it as an environmental blocker rather than rebooting hopefully.

PART



Build the guest

Acquire a verified image, size the machine for a 7.73 GB host, and drive an unattended install past the one prompt that silently breaks every naive attempt.

§5 Phase 4 — Image acquisition and verification

§5.1 Build-path decision: Path B vs Path A

§5.2 Constrained resource sizing profile

§6 Phase 5 — VM creation and unattended install

§6.2 The uppercase-username problem

§6.5 Monitoring, polling and stall diagnosis

SECTION 5 · PHASE 4

Image acquisition

Choose the build path deliberately, discover the current release rather than hardcoding a version, and never install an image whose checksum you have not personally verified.

INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
VirtualBox installed (A1), free space re-verified after teardown.	- Select Path B or Path A - Resolve current ISO filename - Download via BITS - Verify SHA256 against upstream	<code>\$Root\cache\kali-...-installer-amd64.iso</code> - Recorded SHA256	Computed hash equals the published hash. Any mismatch deletes the file and halts.

5.1 Build-path decision

TABLE 5.1 — PATH COMPARISON

DIMENSION	PATH B — INSTALLER ISO PRIMARY	PATH A — PREBUILT IMAGE FALLBACK
Method	Unattended install from <code>kali-linux-<ver>-installer-amd64.iso</code>	Import Kali's official VirtualBox <code>.7z</code> appliance
Satisfies "from scratch"	Yes, fully — every package chosen by this directive	Partially — the image is pre-built by Kali
Peak disk	ISO ~4 GB + guest ~20 GB	Archive ~4 GB + expanded ~13 GB
Wall time	40–70 min on this host	15–25 min
Credential handling	Set natively during install	Post-boot rename via <code>guestcontrol</code> (ships as <code>kali / kali</code>)
Failure modes	Preseed drift, installer prompt stalls	Fewer — but less control over the result

GATE **PATH SELECTION RULE**

Use Path B. Fall back to Path A only if Path B fails twice *for the same root cause*, or if gate G4 leaves under 45 GB available. **Announce any fallback explicitly** — the operator asked for a machine built from scratch, and a silent substitution violates rule R5 even when the result boots.

5.2 Resource sizing for this host

TABLE 5.2 — SIZING PROFILES

SETTING	CONSTRAINED USE THIS	STANDARD (≥ 16 GB HOST)
Guest RAM	3072 MB	6144 MB
vCPU	2	4
VRAM	128 MB	128 MB
Disk (dynamic VDI)	60 GB max, ~18 GB actual	100 GB
Metapackage	kali-linux-core + kali-desktop-xfce	kali-linux- default
Audio	disabled	enabled

CAUTION THE 50 % RULE

Never allocate more than 50 % of host RAM to the guest. At 7.73 GB total, 3072 MB is already 39 %. Tell the operator to close browsers and idle MCP servers before starting the VM.

WHY XFCE AND NOT THE DEFAULT METAPACKAGE

`kali-linux-default` pulls several gigabytes of tooling this build does not need and cannot comfortably hold. `kali-linux-core` plus the Xfce desktop produces a working graphical Kali in roughly 18 GB — and anything missing is one `apt install` away, inside the guest, by the operator.

5.3 Download with checksum verification — mandatory

Discover the current release rather than hardcoding a version: **Kali ships quarterly and stale filenames 404.**

Resolve the name from the directory listing, download with BITS so the transfer survives a network blip, then verify.

PWSH writes and verifies `$Root\cache\<iso>`

~15-25 min · ~4 GB

```
$base = 'https://cdimage.kali.org/current/'
$listing = (Invoke-WebRequest $base -UseBasicParsing).Content
$iso = ([regex]'kali-linux-[\d.]+-installer-amd64\.iso').Match($listing).Value
if (-not $iso) { throw 'GATE-P4: Could not resolve current Kali installer ISO name.' }

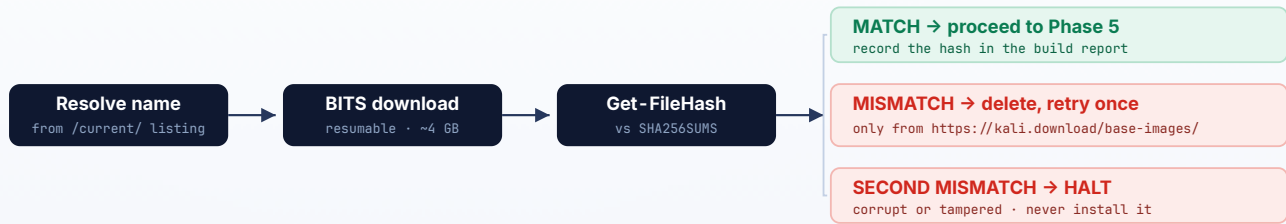
$isoPath = Join-Path "$Root\cache" $iso
Start-BitsTransfer -Source "$base$iso" -Destination $isoPath -DisplayName 'Kali ISO'

# Verify against the published SHA256SUMS
$sums = (Invoke-WebRequest "${base}SHA256SUMS" -UseBasicParsing).Content
$expected = ($sums -split "`n" | Where-Object { $_ -match [regex]::Escape($iso) }) -split '\s+' | Select-Object -First 1
$actual = (Get-FileHash $isoPath -Algorithm SHA256).Hash.ToLower()

if ($actual -ne $expected.ToLower()) {
    Remove-Item $isoPath -Force
    throw "GATE-P4: Checksum mismatch. Expected $expected, got $actual. Image deleted."
}

Write-Host "ISO verified: $iso" -ForegroundColor Green
```


FIGURE 5.1 – VERIFICATION PATH: THREE OUTCOMES, ONE ACCEPTABLE



A checksum mismatch is a hard stop, not a retry loop. Delete the file and report. **Never proceed with an unverified system image** — the whole security value of the build rests on this one comparison.

5.4 If the download is interrupted

A 4 GB transfer on a laptop will occasionally be cut short. BITS was chosen precisely so that this is a resume, not a restart — but only if you ask it to resume rather than starting a new job.

PWSH resume rather than re-download

varies

```
$job = Get-BitsTransfer -Name 'Kali ISO' -ErrorAction SilentlyContinue
if ($job) {
    if ($job.JobState -eq 'Suspended') { Resume-BitsTransfer $job -Asynchronous }
    Write-Host ("Resumed at {0:N1} of {1:N1} GB" -f ($job.BytesTransferred/1GB), ($job.BytesTotal/1GB))
} else {
    Start-BitsTransfer -Source "$base$iso" -Destination $isoPath -DisplayName 'Kali ISO'
}
```

CAUTION A PARTIAL FILE STILL HASHES

`Get-FileHash` happily hashes a truncated download and produces a mismatch that looks exactly like tampering. Before concluding "corrupt or tampered" (risk R-05), confirm the file size matches the size the mirror advertises. Only then apply the retry-then-halt rule.

SECTION 6 · PHASE 5 · THE LONG PHASE

VM creation and unattended install

Create the machine, write a preseed that survives Debian-installer's validation rules, launch the install, and poll it to completion rather than sleeping blindly.

INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
Verified ISO, VirtualBox 7.x, sizing profile from §5.2.	<code>createvm</code> / <code>modifyvm</code> / <code>storageattach</code> - Write preseed template <code>unattended install --start-vm=gui</code> - Poll guest properties	- Registered VM <code>kali</code> (A2) <code>kali.vdi</code> - Running guest (A4)	<code>GuestInfo/OS/Product</code> matches <code>Linux</code> within the 90-minute deadline.

6.1 Create and configure

PWSH
creates and registers VM `kali` — criterion A2
~30 s

```

$VM = 'kali'
& $VBoxManage createvm --name $VM --ostype Debian_64 --register

& $VBoxManage modifyvm $VM `
  --memory 3072 --cpus 2 --vram 128 `
  --graphicscontroller vmsvg --accelerate3d off `
  --firmware efi --ioapic on --rtcuseutc on `
  --nested-hw-virt off --paravirtprovider kvm `
  --audio-driver none `
  --clipboard-mode bidirectional --draganddrop bidirectional `
  --nic1 nat --nictype1 virtio `
  --boot1 dvd --boot2 disk --boot3 none --boot4 none `
  --description 'Kali Linux lab guest — built by agent directive'

$vmDir = Split-Path (& $VBoxManage showvminfo $VM --machinereadable |
  Select-String '^CfgFile=' | ForEach-Object { ($_.split '"')[1] })

& $VBoxManage createmedium disk --filename "$vmDir\$VM.vdi" --size 61440 --variant Standard
& $VBoxManage storagectl $VM --name 'SATA' --add sata --controller IntelAhci --portcount 2
& $VBoxManage storageattach $VM --storagectl 'SATA' --port 0 --device 0 --type hdd `
  --medium "$vmDir\$VM.vdi" --nonrotational on --discard on
& $VBoxManage storageattach $VM --storagectl 'SATA' --port 1 --device 0 --type dvddrive --medium $isoPath

```

NOTE ON --OSTYPE

Run `VBoxManage list ostypes | Select-String Debian` and select the highest-numbered `Debian*_64` your VirtualBox build offers. Kali is Debian-derived; VirtualBox does not ship a dedicated Kali profile, and picking a stale Debian generation costs you sensible controller defaults.

WHY THESE FLAGS

- ◆ `--firmware efi` — matches the preseed's EFI partitioning recipe
- ◆ `--nictype1 virtio` — paravirtualised NIC, far less CPU than e1000
- ◆ `--discard on` — TRIM passthrough keeps the dynamic VDI from growing forever
- ◆ `--audio-driver none` — one less device on a memory-constrained host
- ◆ `--accelerate3d off` — 3D acceleration is the most common cause of Xfce session crashes under VMSVGA

6.2 The uppercase-username problem — read before writing the preseed

DESTRUCTIVE THIS IS THE SINGLE MOST COMMON FAILURE IN THIS BUILD

Debian-installer's `user-setup` validates usernames against a **lowercase-only** regex. Preseeding Kali with a capital `K` causes the installer to reject the value and stall at an interactive prompt — which silently breaks unattended installation. The VM stays in state `running` forever and no guest property ever appears.

FIGURE 6.1 — TWO WAYS TO PRESEED A CAPITALISED LOGIN; ONLY ONE FINISHES

NAIVE — PRESEED THE NAME YOU WANT

```
d-i passwd/username
string Kali
preseed value
```

```
user-setup validates
^[a-z][[-a-z0-9_]]*$
capital K rejected
```

```
installer falls back to
an interactive prompt
blue screen, waiting
```

**Unattended install
never completes**

THIS DIRECTIVE — CREATE LOWERCASE, RENAME AFTER user-setup

```
d-i passwd/username
string kali
accepted
```

```
user-setup creates
uid 1000 · group kali
install continues
```

```
late_command:
usermod --badname -l Kali
fallback: sed /etc/passwd
```

```
id Kali →
uid=1000(Kali)
gid=1000(kali)
```

Rename the user **only** — leaving the primary group lowercase avoids cascading breakage in `/etc/group`, `/etc/gshadow`, `/etc/subuid`, `/etc/subgid` and `sudoers`.

The resulting identity `uid=1000(Kali) gid=1000(kali)` is correct and stable — a mixed-case user in a lowercase group is entirely normal on Linux.

Resolution, in order of preference

- 1 **Preferred:** `usermod --badname` (shadow ≥ 4.13, present in current Kali).
- 2 **Fallback:** direct edit of `/etc/passwd` and `/etc/shadow`.
- 3 **Verify either way** with `id Kali`. If verification fails, *the build fails* — do not report a partial success.

CAUTION TELL THE OPERATOR PLAINLY

Linux logins are case-sensitive. The login is `Kali`, not `kali`. Put this sentence in the handover summary and on the Control Center window — it is the difference between "the VM works" and a support call ten minutes after handover.

6.3 Preseed template

Write to `$Root\cache\kali-preseed.cfg`. Every block is load-bearing; the annotations name what breaks if you drop it.

PRESEED file `$Root\cache\kali-preseed.cfg`

debian-installer syntax — not PowerShell

```
d-i debian-installer/locale      string en_US.UTF-8
d-i keyboard-configuration/xkb-keymap select us
d-i netcfg/choose_interface     select auto
d-i netcfg/get_hostname        string kali
d-i netcfg/get_domain          string local
d-i mirror/country              string manual
d-i mirror/http/hostname        string http.kali.org
d-i mirror/http/directory      string /kali
d-i mirror/http/proxy           string
d-i clock-setup/utc             boolean true
d-i time/zone                   string UTC
d-i clock-setup/ntp             boolean true

# --- Accounts: root disabled, sudo-capable user created lowercase -----
d-i passwd/root-login          boolean false
d-i passwd/make-user           boolean true
d-i passwd/user-fullname       string Kali
d-i passwd/username            string kali
d-i passwd/user-password       password bot
d-i passwd/user-password-again password bot
d-i user-setup/allow-password-weak boolean true
d-i user-setup/encrypt-home    boolean false

# --- Partitioning: whole disk, single partition, no LVM -----
d-i partman-auto/method      string regular
d-i partman-auto/choose_recipe select atomic
d-i partman-partitioning/confirm_write_new_label boolean true
d-i partman/choose_partition   select finish
d-i partman/confirm            boolean true
d-i partman/confirm_nooverwrite boolean true
d-i partman-efi/non_efi_system boolean true

# --- Package selection (constrained profile) -----
tasksel tasksel/first          multiselect
d-i pkgsel/include              string kali-linux-core kali-desktop-xfce sudo openssh-client
d-i pkgsel/upgrade              select full-upgrade
popularity-contest popularity-contest/participate boolean false

# --- Bootloader -----
d-i grub-installer/only_debian  boolean true
d-i grub-installer/bootdev      string default

# --- Post-install: enforce the uppercase login -----
d-i preseed/late_command string \
in-target sh -c 'usermod --badname -l Kali kali \
|| (sed -i "s/^kali:/Kali:/" /etc/passwd /etc/shadow)' ; \
in-target sh -c 'usermod --badname -d /home/Kali -m Kali \
|| (mv /home/kali /home/Kali; sed -i "s#:/home/kali:##:/home/Kali:#" /etc/passwd)' ; \
in-target chown -R Kali:kali /home/Kali ; \
in-target sh -c 'echo "Kali ALL=(ALL:ALL) ALL" > /etc/sudoers.d/kali-user; chmod 440 /etc/sudoers.d/kali-user'
; \
in-target id Kali > /target/var/log/agent-username-check.log 2>&1

d-i finish-install/reboot_in_progress note
```

TABLE 6.1 — LOAD-BEARING PRESEED KEYS

KEY	WHAT BREAKS WITHOUT IT
passwd/root-login false	Root gets a password and the sudo model is inconsistent with Kali's default posture.
user-setup/allow-password-weak	A three-character password is rejected and the installer prompts — an unattended stall.
partman-efi/non_efi_system	EFI firmware (set in §6.1) plus a non-EFI recipe halts partitioning with a confirmation dialog.
partman/confirm_nooverwrite	The "write changes to disk?" dialog appears and waits forever.
pkgsel/include	You get a minimal console system with no desktop — criterion A4 cannot pass.
preseed/late_command	The account stays <code>kali</code> — criterion A5 fails.
finish-install/reboot_in_progress	The installer waits for acknowledgement at the very last step.

6.4 Launch the unattended install

PWSH starts the guest in GUI mode so a stall is visible
 returns immediately

```
& $VBoxManage unattended install $VM `
  --iso=$isoPath `
  --user=kali --password=bot --full-user-name='Kali' `
  --hostname=kali.local `
  --locale=en_US --country=US --time-zone=UTC `
  --script-template="$Root\cache\kali-preseed.cfg" `
  --install-additions `
  --start-vm=gui
```

CAUTION
--HOSTNAME
MUST CONTAIN A DOT

VirtualBox rejects a bare hostname outright. Use `kali.local`; the guest's own `/etc/hostname` still ends up as `kali` from the preseed.

VERIFY
--INSTALL-ADDITIONS

Attaches the bundled `VBoxGuestAdditions.iso` and installs it at the end of the run — this is what satisfies **criterion A6**. Without it the host never learns the guest's OS or IP.

6.5 Monitor to completion

Poll rather than sleeping blindly. Expect 40–70 minutes on this host; the mirror upgrade step dominates and produces long stretches with no visible change.

PWSH poll loop with a hard 90-minute deadline
 40–70 min typical

```
$deadline = (Get-Date).AddMinutes(90)
do {
  Start-Sleep 30
  $state = (& $VBoxManage showvminfo $VM --machinereadable |
    Select-String '^VMState=') -replace '.*(.*)'.*', '$1'
  $os = & $VBoxManage guestproperty get $VM /VirtualBox/GuestInfo/OS/Product 2>$null
  Write-Host "[$(Get-Date -f HH:mm:ss)] state=$state guest=$os"
} until ($os -match 'Linux' -or (Get-Date) -gt $deadline)
```

```
if ($os -notmatch 'Linux') { throw 'GATE-P5: Install did not complete within 90 minutes.' }
```

Stall diagnosis

If the state stays `running` but no guest properties ever appear, the installer is almost certainly sitting on an interactive prompt. **Take a screenshot and read it — do not guess.**

PWSH capture what the guest is actually showing

instant

```
& $VBoxManage controlvm $VM screenshotpng "$Root\logs\stall-$(Get-Date -f HHmmss).png"
```

TABLE 6.2 — READING THE STALL SCREENSHOT

WHAT THE SCREEN SHOWS	DIAGNOSIS AND ACTION
A prompt on the user account screen	Confirms the §6.2 username issue. Correct the preseed and rebuild — do not answer the prompt by hand, or the next run stalls identically.
"Write the changes to disks?"	A partman confirmation key is missing. Check <code>partman/confirm*</code> in the template.
Mirror or network configuration screen	The mirror is unreachable. Rule R5 applies: report it; do not substitute a different mirror on your own initiative.
A black screen, no cursor, state <code>running</code>	Usually 3D acceleration under VMSVGA. Power off, confirm <code>--accelerate3d off</code> , restart.
GRUB rescue prompt	Bootloader install failed. Check <code>grub-installer/bootdev</code> and that the ISO is no longer first in the boot order.

6.6 What healthy progress looks like

The poll loop prints one line every thirty seconds. Long runs of identical lines are normal — the installer is downloading and unpacking the mirror upgrade, which produces no observable change from the host side.

CONSOLE representative output from §6.5

abridged

```
[10:02:31] state=running  guest=
[10:03:01] state=running  guest=
[10:24:31] state=running  guest=          <- mirror full-upgrade, ~25 min of this is expected
[10:51:01] state=running  guest=
[10:58:31] state=powersoff guest=          <- first reboot after install; VirtualBox may report powersoff
briefly
[10:59:31] state=running  guest=
[11:04:01] state=running  guest=Value: Kali GNU/Linux  <- Guest Additions are up: A6 satisfied, loop exits
```

OBSERVATION	INTERPRETATION
Empty <code>guest=</code> for 20–30 minutes	Normal during the mirror upgrade. Do not intervene before the deadline.
A single <code>powersoff</code> then <code>running</code>	The post-install reboot. Expected exactly once.
<code>aborted</code>	The VM process died — usually host memory pressure. Check free RAM, reduce the guest to 2048 MB, restart the phase.

OBSERVATION	INTERPRETATION
Empty <code>guest=</code> at 85 minutes	Take the screenshot now, before the deadline throws, so the diagnosis is in the log alongside the failure.

Guest properties worth polling

Guest properties are the host's only structured view into a running guest. They appear one at a time as the Additions service initialises, so treat an absent key as "not yet" rather than "broken".

KEY	TELLS YOU
<code>/VirtualBox/GuestInfo/OS/Product</code>	The Additions service is up and the guest is Linux — criterion A6, and the exit condition of the poll loop.
<code>/VirtualBox/GuestInfo/OS/Release</code>	Kernel release. Useful when diagnosing a failed Additions build against missing headers.
<code>/VirtualBox/GuestInfo/Net/0/V4/IP</code>	The guest's NAT address, typically <code>10.0.2.15</code> . Its presence proves networking came up inside the guest.
<code>/VirtualBox/GuestAdd/Version</code>	Additions version. If it lags the host's VirtualBox version, guest control calls may behave inconsistently.
<code>/VirtualBox/GuestInfo/OS/LoggedInUsers</code>	Non-zero once someone logs in — a quiet way to confirm the operator reached the desktop during the smoke test.
<code>/VirtualBox/HostInfo/VBoxVer</code>	Reported from the host side; handy to include verbatim in the build report next to A1.

NOTE ENUMERATE BEFORE YOU GUESS

`VBoxManage guestproperty enumerate kali` lists every key the guest currently publishes, with timestamps. When a specific `get` returns `No value set!`, enumerate first: the difference between "the Additions are not running" and "that key does not exist on this Additions version" changes the fix entirely.

PART

IV

Prove it & hand it over

A build that cannot be verified has not finished. Prove each criterion by command, take the rollback snapshot, then give the operator a surface they can use without a terminal.

§7 Phase 6 — Post-install validation and baseline

§7.3 In-guest recovery when A5 fails

§9 Failure playbook

§8 Phase 7 — Desktop control application

§8.4 Smoke test — the only proof for A8

§10 Final report format

SECTION 7 · PHASE 6

Post-install validation and baseline

Detach the installer, prove the credentials and the Guest Additions, then freeze a snapshot the operator can always fall back to.

INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
Guest reporting <code>Linux</code> via guest properties.	- Detach installer media - Fix boot order - Prove A5, A6 - Take <code>clean-baseline</code>	- Snapshot <code>clean-baseline</code> (A9) - Captured verification output	EXIT CRITERION A4, A5, A6 and A9 all proved by command output in a single pass.

7.1 Detach, verify, snapshot

PWSH

proves A5, A6 and creates A9

~5 min incl. clean shutdown

```

# Detach installer media so the guest boots from disk
& $VBoxManage storageattach $VM --storagectl 'SATA' --port 1 --device 0 --medium emptydrive
& $VBoxManage modifyvm $VM --boot1 disk --boot2 none

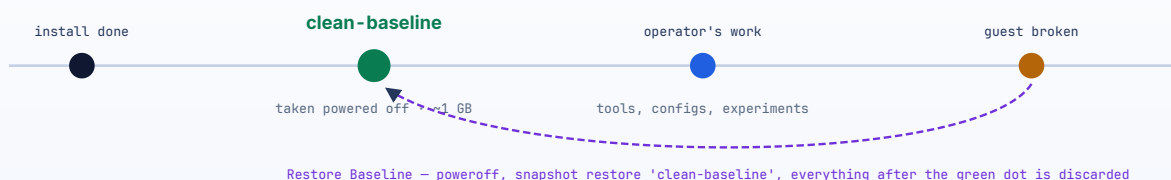
# A5 - credential proof
& $VBoxManage guestcontrol $VM run --username Kali --password bot `
  --exe /usr/bin/id --wait-stdout -- id Kali
# Expect: uid=1000(Kali) gid=1000(kali) groups=...

# A6 - Guest Additions live
& $VBoxManage guestproperty get $VM /VirtualBox/GuestInfo/OS/Product
& $VBoxManage guestproperty get $VM /VirtualBox/GuestInfo/Net/0/V4/IP

# A9 - rollback point, taken from a cleanly powered-off guest
& $VBoxManage controlvm $VM acpipowerbutton
Start-Sleep 30
& $VBoxManage snapshot $VM take 'clean-baseline' `
  --description 'First good boot: Kali/bot, Guest Additions, no user changes.'
```

7.2 Snapshot lifecycle

FIGURE 7.1 — `CLEAN-BASELINE` AND WHAT IT PROTECTS



Take the snapshot from a *cleanly powered-off* guest, not a saved state: a powered-off baseline restores predictably on any later VirtualBox version.

7.3 If A5 fails — recover in-guest, do not rebuild

The username rename did not take. Two hours of install do not need to be repeated for a one-line fix.

PWSH in-guest recovery — log in as the lowercase account and rename

~1 min

```
& $VBoxManage guestcontrol $VM run --username kali --password bot `
  --exe /bin/su --wait-stdout -- su -c 'usermod --badname -l Kali kali'

# then re-prove A5
& $VBoxManage guestcontrol $VM run --username Kali --password bot `
  --exe /usr/bin/id --wait-stdout -- id Kali
```

If the home directory did not move with the account, follow with `usermod --badname -d /home/Kali -m Kali` and `chown -R Kali:kali /home/Kali`. Re-take `clean-baseline` after any recovery — a baseline that predates the fix is worse than none, because restoring it silently reintroduces the failure.

7.4 Security posture of the finished guest

DESTRUCTIVE SECURITY NOTE FOR THE OPERATOR — SAY THIS OUT LOUD

`bot` is a three-character password. **It is acceptable only because this guest sits behind NAT with no inbound path.** Keep it that way — no port forwarding, no bridged adapter, no SSH exposure. If the VM is ever moved onto a bridged or public network, **change the password first.**

TABLE 7.1 — POSTURE SUMMARY AS DELIVERED

CONTROL	STATE	RATIONALE / WHAT WOULD CHANGE IT
Inbound network path	none	NAT only, no port forwards. Adding one invalidates the password rationale above.
Root login	disabled	<code>passwd/root-login false</code> ; escalation is via <code>sudo</code> as <code>Kali</code> .
Password strength	weak	Deliberate, scoped, and documented. Change it before any network change.
SSH server	not installed	Only <code>openssh-client</code> is in <code>pkgset/include</code> .
Clipboard / drag-drop	bidirectional	Convenience feature; it is a host↔guest data path. Disable it if the guest will handle untrusted samples.
Host firewall	untouched	Non-goal §0.2. Nothing in this build modifies host rules.
Host VBS / HVCI	untouched	Rule R6 and gate G5. Only the operator may change this.

Desktop control application

Criterion A8 requires start, configure and shut down from a desktop-launched manager. Deliver both layers: a purpose-built control panel as the primary surface, and the full VirtualBox Manager reachable from it for advanced configuration.

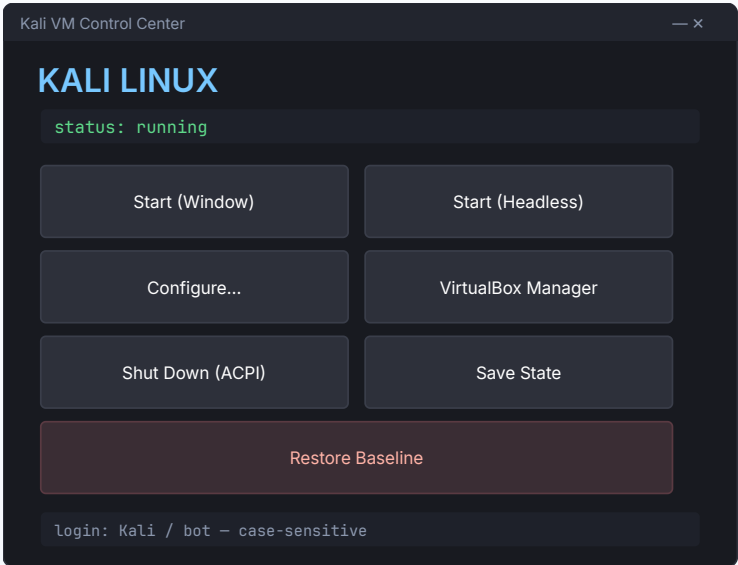
INPUTS	ACTIONS	OUTPUTS	EXIT CRITERION
Elevated session (R1), VM registered, snapshot present.	- Write <code>KaliControlCenter.ps1</code> - Write silent launcher <code>launch.vbs</code> - Create desktop shortcut - Walk the operator through the smoke test	<code>Kali VM Control Center.lnk</code> (A7) - Working control panel (A8)	Shortcut exists and the operator confirms the §8.4 smoke test end to end.

8.1 Control Center — interface specification

One fixed-size window, no maximise, dark surface, seven actions and a live status line that polls every two seconds. Nothing in it requires the operator to know that `VBoxManage` exists.

ELEMENT	BEHAVIOUR
Status line	2 s timer, colour-coded state
Start (Window)	<code>startvm --type gui</code>
Start (Headless)	<code>startvm --type headless</code>
Configure...	<code>--startvm-settings</code>
VirtualBox Manager	full GUI, advanced use
Shut Down (ACPI)	<code>controlvm acpipowerbutton</code>
Save State	<code>controlvm savestate</code>
Restore Baseline	confirm dialog → poweroff → restore

FIGURE 8.1 — CONTROL CENTER, AS DELIVERED



Status colours: running · saved / paused · poweroff / aborted · grey for anything else.

```
PWSH file C:\ProgramData\KaliVMBuild\KaliControlCenter.ps1 WinForms · no elevation to run

Add-Type -AssemblyName System.Windows.Forms, System.Drawing
$ErrorActionPreference = 'Stop'
```

```

$VM = 'kali'
$VBox = (Get-ItemProperty 'HKLM:\SOFTWARE\Oracle\VirtualBox').InstallDir
$VBM = Join-Path $VBox 'VBoxManage.exe'
$GUI = Join-Path $VBox 'VirtualBox.exe'

function Get-VMState {
    try { ((& $VBM showvminfo $VM --machinereadable 2>$null |
        Select-String '^VMState=') -replace '.*"(.*)"'.*, '$1') }
    catch { 'unavailable' }
}

$form = New-Object Windows.Forms.Form
$form.Text = 'Kali VM Control Center'
$form.Size = New-Object Drawing.Size(430, 330)
$form.StartPosition = 'CenterScreen'
$form.FormBorderStyle = 'FixedSingle'
$form.MaximizeBox = $false
$form.BackColor = [Drawing.Color]::FromArgb(24,26,32)

$lblTitle = New-Object Windows.Forms.Label
$lblTitle.Text = 'KALI LINUX'
$lblTitle.Font = New-Object Drawing.Font('Segoe UI Semibold',16)
$lblTitle.ForeColor = [Drawing.Color]::FromArgb(120,200,255)
$lblTitle.Location = New-Object Drawing.Point(20,15)
$lblTitle.AutoSize = $true
$form.Controls.Add($lblTitle)

$lblState = New-Object Windows.Forms.Label
$lblState.Location = New-Object Drawing.Point(22,50)
$lblState.Size = New-Object Drawing.Size(370,22)
$lblState.Font = New-Object Drawing.Font('Consolas',10)
$lblState.ForeColor = [Drawing.Color]::Gainsboro
$form.Controls.Add($lblState)

function New-ActionButton($text,$x,$y,$w,$action) {
    $b = New-Object Windows.Forms.Button
    $b.Text = $text
    $b.Location = New-Object Drawing.Point($x,$y)
    $b.Size = New-Object Drawing.Size($w,42)
    $b.FlatStyle = 'Flat'
    $b.BackColor = [Drawing.Color]::FromArgb(45,48,58)
    $b.ForeColor = [Drawing.Color]::White
    $b.Font = New-Object Drawing.Font('Segoe UI',9.5)
    $b.Add_Click($action)
    $form.Controls.Add($b)
    $b
}

New-ActionButton 'Start (Window)' 22 85 180 { & $VBM startvm $VM --type gui } | Out-Null
New-ActionButton 'Start (Headless)' 212 85 180 { & $VBM startvm $VM --type headless } | Out-Null
New-ActionButton 'Configure...' 22 135 180 { Start-Process $GUI -ArgumentList '--startvm-settings',$VM } | Out-Null
New-ActionButton 'VirtualBox Manager' 212 135 180 { Start-Process $GUI } | Out-Null
New-ActionButton 'Shut Down (ACPI)' 22 185 180 { & $VBM controlvm $VM acpipowerbutton } | Out-Null
New-ActionButton 'Save State' 212 185 180 { & $VBM controlvm $VM savestate } | Out-Null
New-ActionButton 'Restore Baseline' 22 235 370 {
    if ([Windows.Forms.MessageBox]::Show(
        'Discard all changes and restore the clean-baseline snapshot?',
        'Confirm rollback','YesNo','Warning') -eq 'Yes') {
        & $VBM controlvm $VM poweroff 2>$null
        Start-Sleep 3
        & $VBM snapshot $VM restore 'clean-baseline'
    }
} | Out-Null

```

```
$timer          = New-Object Windows.Forms.Timer
$timer.Interval = 2000
$timer.Add_Tick({
    $s = Get-VMState
    $lblState.Text = "  status:  $s"
    $lblState.ForeColor = switch -Regex ($s) {
        'running'          { [Drawing.Color]::LimeGreen }
        'saved|paused'     { [Drawing.Color]::Gold }
        'poweroff|aborted' { [Drawing.Color]::IndianRed }
        default            { [Drawing.Color]::Gray }
    }
})
$timer.Start()

[void]$form.ShowDialog()
```

8.2 Silent launcher

A `.lnk` pointing at `powershell.exe` flashes a console window every launch. Wrap it.

```
VBSCRIPT file C:\ProgramData\KaliVMBuild\launch.vbs window style 0 = hidden

Set sh = CreateObject("WScript.Shell")
sh.Run "pwsh.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File
""C:\ProgramData\KaliVMBuild\KaliControlCenter.ps1""", 0, False
```

Fall back to `powershell.exe` if `pwsh.exe` is not on PATH for the launching user — the agent's elevated session and the operator's interactive session do not necessarily share a PATH.

8.3 Create the desktop icon — criterion A7

```
PWSH creates Desktop\Kali VM Control Center.lnk instant

$desktop = [Environment]::GetFolderPath('Desktop')
$lnkPath = Join-Path $desktop 'Kali VM Control Center.lnk'

$wsh = New-Object -ComObject WScript.Shell
$sc = $wsh.CreateShortcut($lnkPath)
$sc.TargetPath = "$env:SystemRoot\System32\wscript.exe"
$sc.Arguments = '"C:\ProgramData\KaliVMBuild\launch.vbs"'
$sc.WorkingDirectory = 'C:\ProgramData\KaliVMBuild'
$sc.IconLocation = "$((Get-ItemProperty 'HKLM:\SOFTWARE\Oracle\VirtualBox').InstallDir)VirtualBox.exe,0"
$sc.Description = 'Start, configure, and shut down the Kali Linux virtual machine'
$sc.WindowStyle = 1
$sc.Save()

if (-not (Test-Path $lnkPath)) { throw 'GATE-P7: Desktop shortcut was not created.' }
```

KNOWN-FOLDER MOVE

Use

```
[Environment]::GetFolderPath('Desktop')
```

— **never** a hardcoded `C:\Users\...\Desktop`.
It resolves correctly under OneDrive Known Folder Move.

ALL USERS

Place a duplicate in
`$env:PUBLIC\Desktop`
(elevation required)
only if the operator
wants it visible to every
account on the
machine.

ICON NOT APPEARING

Refresh the shell: `Stop-Process -Name explorer -Force` — Explorer restarts automatically. For a Start Menu entry, write the same `.lnk` to
`$env:APPDATA\Microsoft\Windows\Start Menu\Programs`.

8.4 Smoke test — criterion A8

Walk the operator through this sequence and **confirm each step aloud**. A8 stays `null` in the report until they have done it.

- ☐ Double-click the desktop icon — the panel opens and shows status `poweroff`.
- ☐ Click **Start (Window)** — a VirtualBox window appears and the guest begins to boot.
- ☐ The guest reaches the Kali graphical login screen (this also re-confirms A4).
- ☐ Log in as `Kali` / `bot` — *capital K* (this re-confirms A5 interactively).
- ☐ Return to the panel — status now reads `running` in green.
- ☐ Click **Shut Down (ACPI)** — the guest halts cleanly, no forced power-off.
- ☐ Status returns to `poweroff` within roughly 30 seconds.

Only then declare the build complete.

8.5 What to say at handover

The last thing the agent does in Phase 7 is talk. Cover these six points, in this order, in plain language — no identifiers, no phase numbers, nothing the operator has to look up.

- 1 **What was removed.** Name the VMs and files deleted in Phase 2 and the space reclaimed. If anything was protected and skipped, say what and why.
- 2 **How to start it.** The desktop icon, and that the first boot takes about a minute.
- 3 **The login, spelled out.** "Capital K — `Kali`, password `bot`. Linux is case-sensitive, so lowercase will not work."
- 4 **The rollback.** What *Restore Baseline* does, and that it discards everything saved inside the guest since the build.
- 5 **The security condition.** The password is safe only behind NAT. No bridged networking, no port forwarding, no SSH — change the password first if that ever changes.
- 6 **The memory cost.** 3 GB of 7.7 GB; close browsers before starting it.

NOTE HANDOVER IS PART OF THE BUILD

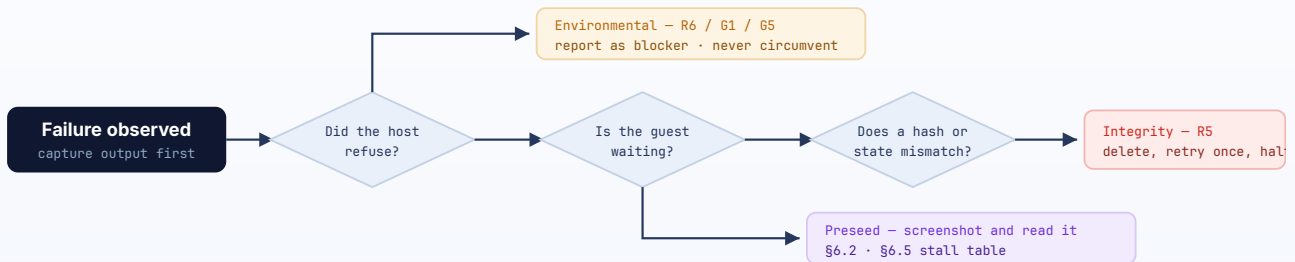
A correctly built machine that the operator cannot log into has failed just as completely as one that never booted. Treat the six points above as deliverables, not pleasantries — and put them in the written summary as well as saying them, because nobody remembers a spoken password rule a week later.

SECTION 9 · REFERENCE

Failure playbook

Match the symptom, confirm the root cause, apply the stated action. Do not skip the confirmation step — two of these symptoms have look-alikes with entirely different fixes.

FIGURE 9.1 – TRIAGE: THREE QUESTIONS THAT CLASSIFY ALMOST EVERY FAILURE



If all three answers are "no", the failure is a resource problem — memory, disk or CPU contention — and the last two rows of the table below apply.

TABLE 9.1 – SYMPTOM → ROOT CAUSE → ACTION

SYMPTOM	ROOT CAUSE	ACTION
VERR_VMX_NO_VMX VERR_SVM_DISABLED	VT-x off in firmware	Halt. Send the operator to BIOS — gate G1 . Re-run Phase 1 afterwards, not Phase 3.
VERR_VMX_IN_VMX_ROOT_MODE	Hyper-V / VBS owns the CPU	Report the conflict and its performance cost. Await instruction — gate G5 . Do not disable VBS yourself.
Install stalls at a blue prompt screen	Preseed key rejected — usually the uppercase username	Screenshot, read the prompt, correct the preseed, rebuild. See §6.2. Do not answer the prompt by hand.
Guest boots straight back into the installer	Installer ISO still attached and first in boot order	Apply the §7.1 detach block: <code>storageattach ... --medium emptydrive</code> then <code>--boot1 disk</code> .
id Kali → "no such user"	<code>late_command</code> rename failed	Apply the in-guest recovery in §7.3. Re-take <code>clean-baseline</code> afterwards.
Guest Additions absent (A6 fails)	Kernel headers missing during install	In-guest: <code>sudo apt update && sudo apt install -y linux-headers-\$(uname -r)</code> , then rerun the Additions installer.
Host thrashes / becomes unresponsive	3 GB guest on a 7.73 GB host with browsers open	Drop guest RAM to 2048 MB, use <code>kali-linux-core</code> only, close MCP servers before starting.
VirtualBox install fails with a policy/signature error	MDM driver-installation restriction	Report as an environmental blocker. Do not circumvent — rule R6 .

SYMPTOM	ROOT CAUSE	ACTION
Checksum mismatch	Corrupt or tampered download	Delete, retry once from <code>https://kali.download/base-images/</code> . A second failure is a halt.
E_ACCESSDENIED on guestcontrol	Guest Additions not yet running, or wrong case in the username	Confirm A6 first, then retry with a capital <code>K</code> . Case is the usual culprit.
Run interrupted mid-Phase 5	Session dropped, host slept, MDM restart	Resume from <code>state.json</code> (Appendix D). Never re-run Phase 2 to "start clean".

SECTION 10

Final report format

Emit as `$Root\manifests\build-report.json` and summarize to the operator in prose. The JSON is the record; the prose is the handover.

JSON

file `$Root\manifests\build-report.json`

emitted once, at the end

```
{
  0: 45,
  1: 46,
  2: { 3: 47, 4: 48, 5: 0, 6: 0, 7: 57 },
  8: { 9: [], 10: 0, 11: 0, 12: [] },
  13: { 14: 49, 15: 50, 16: 58 },
  17: { 18: 51, 19: 52, 20: 59 },
  21: { 22: 53, 23: 3072, 24: 2, 25: 60, 26: 54 },
  27: { 28: 55, 29: 60 },
  30: { 31: 56, 32: 61 },
  33: { 34: 62, 35: 63, 36: 64, 37: 65, 38: 66,
      39: 67, 40: 68, 41: 69, 42: 70 },
  43: [],
  44: []
}
```

TABLE 10.1 – FIELD SEMANTICS THAT ARE EASY TO GET WRONG

FIELD	RULE
result	<code>SUCCESS</code> only when A1–A7 and A9 are <code>true</code> and A8 is confirmed. Any <code>false</code> makes it <code>FAILED</code> ; an unconfirmed A8 makes it <code>PARTIAL</code> .
acceptance.A8	<code>null</code> until the operator confirms the smoke test . Never self-certify a criterion that requires human observation.
teardown.protectedSkipped	Always populated when protected candidates were found — even if none were deleted. Silence here reads as "nothing was protected".

VERIFY

THE HANDOVER PARAGRAPH

Alongside the JSON, tell the operator in plain words: what was deleted, what the login is (**capital K**), where the icon is, what the snapshot is for, and the one security condition — no bridged networking while the password is `bot`.

FIELD	RULE
image.verified	<code>true</code> only if the SHA256 comparison ran and matched in this run. A cached ISO from a previous run is re-hashed, not assumed.
warnings	Every constrained-profile decision, every gate that fired, every operator trade-off (G5) goes here as a sentence.
operatorActionsRequired	What the human must still do. Empty array means genuinely nothing — not "nothing I could think of".

CLOSING THE RUN

- ◆ Write `build-report.json`
- ◆ Mark all phases complete in `state.json`
- ◆ `Stop-Transcript` — rule R3, on both the success and the failure path
- ◆ State the total elapsed time and the peak disk used

DESTRUCTIVE END OF DIRECTIVE

A8 is null until the operator confirms the smoke test. Do not self-certify a criterion that requires human observation. A build reported as SUCCESS that the operator has never watched boot is not a success — it is an unverified claim with a green label on it.

10.2 One-pass final verification

Run every acceptance check again, in a fresh shell, immediately before writing the report. Eight of the nine criteria are machine-checkable; the ninth is the operator's.

PWSH emits the `acceptance` block of `build-report.json` ~30 s

```
$VM = 'kali'
$acc = [ordered]@{}
$acc.A1 = (& $VBM --version) -match '^7\.'
$acc.A2 = ((& $VBM list vms) -match '"kali"').Count -eq 1
$acc.A3 = Test-Path "$Root\manifests\teardown-executed.json"
$acc.A4 = ((& $VBM showvminfo $VM --machinereadable) -match '^VMState="running"').Count -gt 0
$acc.A5 = (& $VBM guestcontrol $VM run --username Kali --password bot `
    --exe /usr/bin/id --wait-stdout -- id Kali 2>$null) -match 'uid=\d+\(Kali\)'
$acc.A6 = (& $VBM guestproperty get $VM /VirtualBox/GuestInfo/OS/Product) -match 'Linux'
$acc.A7 = Test-Path (Join-Path ([Environment]::GetFolderPath('Desktop')) 'Kali VM Control Center.lnk')
$acc.A8 = $null # operator-confirmed only — never self-certify
$acc.A9 = (& $VBM snapshot $VM list) -match 'clean-baseline'

$acc | ConvertTo-Json | Tee-Object "$Root\manifests\acceptance.json"
$result = if ($acc.Values -contains $false) { 'FAILED' }
           elseif ($null -eq $acc.A8) { 'PARTIAL' }
           else { 'SUCCESS' }
Write-Host "RESULT: $result" -ForegroundColor Cyan
```

VERIFY A4 IN THE FINAL PASS

A4 asks whether the guest boots to a graphical login unattended. If the VM is powered off at report time — as it will be, since Phase 6 shut it down to take the snapshot — start it once more for this check, or record A4 from the operator's smoke test rather than reporting `false` for a machine that works.

PART



Reference appendices

Everything the spine points at: the commands, the identifiers, the risks, the resume protocol, the vocabulary, and a single page the operator can keep.

-
- | | |
|---|--|
| A VBoxManage command reference card | D Resumable state machine & telemetry |
| B Consolidated index of gates, rules and codes | E Glossary |
| C Risk register and heat map | F Operator quick-start card |

APPENDIX A

VBoxManage command reference card

Every command this directive uses, grouped by what you are trying to find out or change.

`$VBM` is the resolved path to `VBoxManage.exe`.

Inspect

<code>--version</code>	Hypervisor version — A1
<code>list vms</code>	Registered machines — A2
<code>list runningvms</code>	Currently powered on
<code>list hdds</code>	Media registry — teardown post-condition
<code>list ostypes</code>	Valid <code>--ostype</code> values
<code>showvminfo <vm> --machinereadable</code>	Parseable state — A4
<code>guestproperty enumerate <vm></code>	Everything the guest reports
<code>guestproperty get <vm> /VirtualBox/GuestInfo/OS/Product</code>	Guest OS — A6
<code>guestproperty get <vm> /VirtualBox/GuestInfo/Net/0/V4/IP</code>	Guest IP address
<code>snapshot <vm> list</code>	Snapshot tree — A9

Create and configure

<code>createvm --name --ostype --register</code>	Register a new machine register
<code>modifyvm <vm> ...</code>	Hardware profile — §6.1
<code>createmedium disk --filename --size</code>	Dynamic VDI, size in MB
<code>storagectl <vm> --add sata</code>	Add a controller
<code>storageattach <vm> --type hdd dvddrive</code>	Attach disk or ISO
<code>storageattach ... --medium emptydrive</code>	Detach the installer — §7.1

Run and control

<code>startvm <vm> --type gui</code>	Start with a window
<code>startvm <vm> --type headless</code>	Start with no window
<code>controlvm <vm> acpipowerbutton</code>	Clean guest shutdown
<code>controlvm <vm> savestate</code>	Suspend to disk
<code>controlvm <vm> poweroff</code>	Hard stop — last resort
<code>controlvm <vm> screenshotpng <file></code>	Read a stalled installer — §6.5

Guest execution

<code>guestcontrol <vm> run --username --password --exe ... --wait-stdout</code>	Run a command in the guest — A5
<code>guestcontrol <vm> copyto</code>	Push a file into the guest

Snapshots and teardown

<code>snapshot <vm> take <name> --description</code>	Create the baseline — A9
<code>snapshot <vm> restore <name></code>	Roll back — Control Center
<code>unregistervm <vm> --delete</code>	Deregister <i>and</i> delete files
<code>closemedium disk <uuid> --delete</code>	Release orphaned media
<code>extpack install --replace --accept-license=<hash></code>	Extension Pack — §4.2

CAUTION TWO COMMANDS THAT DELETE WITHOUT ASKING

`unregistervm --delete` and `closemedium --delete` remove files from disk immediately. They belong inside the R4 consent block in Phase 2 and nowhere else in this directive.

setproperty machinefolder <path>	Redirect default VM location — §4.3
unattended install <vm> ...	Preseed-driven install — §6.4

A.2 — Parsing `VBoxManage` output in PowerShell

`VBoxManage` emits text, not objects. These four idioms cover every parse this directive performs; copying them is safer than inventing a new regex under time pressure.

PWSH reusable parsing helpers

reference

```
# 1. VM names from: "kali" {8f3c...}
$names = VBoxManage list vms | ForEach-Object { ($_ -split '"')[1] }

# 2. A single machine-readable key: VMState="running"
function Get-VMKey($vm, $key) {
    (& $VBM showvminfo $vm --machinereadable |
     Select-String "^$key=") -replace '.*(.*)'.*', '$1'
}

# 3. Every machine-readable key as a hashtable
$info = @{}
& $VBM showvminfo $vm --machinereadable | ForEach-Object {
    $k, $v = $_ -split '=', 2
    $info[$k] = $v.Trim(' ')
}

# 4. Media UUIDs from: UUID: 8f3c...
$uuids = VBoxManage list hdds | Select-String '^UUID:' | ForEach-Object { ($_ -split '\s+')[1] }
```

PITFALL

WHY IT BITES

Non-zero exit on success	Some subcommands write to stderr on entirely normal paths. Test the output, not <code>\$LASTEXITCODE</code> , unless you have confirmed the exit code is meaningful for that subcommand.
Locale-dependent text	Human-readable output is localised; <code>--machinereadable</code> is not. Always parse the machine-readable form.
Quotes inside names	<code>(\$_ -split '"')[1]</code> breaks on a VM whose name contains a quote. This build names its VM <code>kali</code> , so it is safe here and nowhere else.
Guest properties before Additions	Every <code>guestproperty get</code> returns <code>No value set!</code> until the Additions service starts. Treat that as "not yet", not as failure.

APPENDIX B

Consolidated index of gates, rules and codes

Every identifier in the directive, where it is defined, and what it does when it fires.

GATES — EVALUATED IN PHASE 1

ID	MEANING	§
G1	VT-x / AMD-V enabled in firmware	2.1
G2	SLAT present	2.1
G3	Host RAM ≥ 6 GB	2.1
G4	Target volume ≥ 45 GB free	2.1
G5	Competing hypervisor / VBS conflict	2.1
G6	Home SKU informational	2.1
G7	Elevation available	2.1

STANDING RULES — IN FORCE THROUGHOUT

ID	MEANING	§
R1	Elevation required for Phases 3 and 7	1
R2	Idempotency and resumable state	1
R3	Full transcript, opened and closed	1
R4	Typed <code>DESTROY</code> consent	1, 3.3
R5	No silent substitution	1, 5.1
R6	MDM awareness, never circumvent	1, 4.1
R7	Six-field report per phase	1

ACCEPTANCE CRITERIA — THE DEFINITION OF DONE

ID	PROVED IN	§
A1	Phase 3 — hypervisor functional	4.1
A2	Phase 5 — exactly one VM <code>kali</code>	6.1
A3	Phase 2 — teardown reconciled	3.3
A4	Phase 5 — graphical login unattended	6.5
A5	Phase 6 — <code>Kali</code> / <code>bot</code> works	7.1
A6	Phase 6 — Guest Additions live	7.1
A7	Phase 7 — desktop icon exists	8.3
A8	Phase 7 — operator smoke test	8.4
A9	Phase 6 — <code>cLean-baseline</code> snapshot	7.1

HALT CODES — THROWN, REPORTED VERBATIM

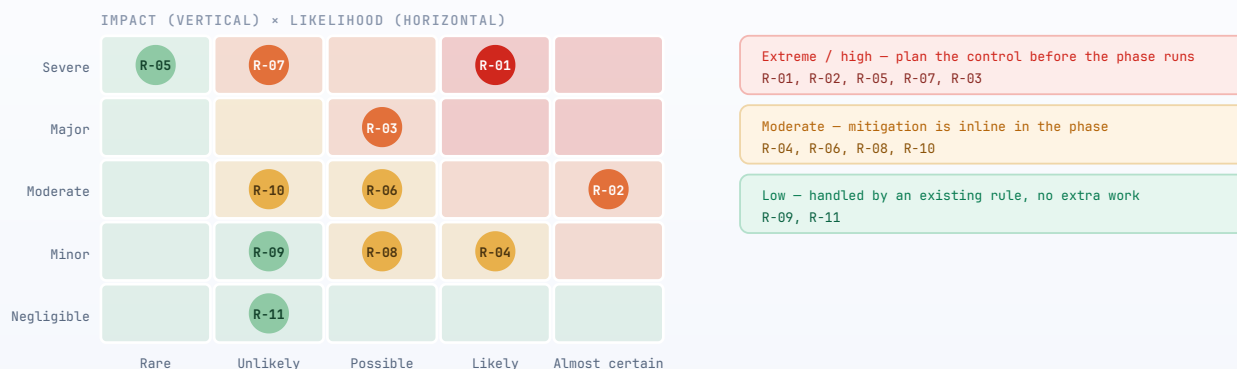
CODE	RAISED WHEN
GATE-R1	Not elevated in a phase that requires it
GATE-R4	Teardown consent not given
GATE-P3	<code>VBoxManage.exe</code> missing after install
GATE-P4	ISO name unresolvable, or checksum mismatch
GATE-P5	Install exceeded the 90-minute deadline
GATE-P7	Desktop shortcut was not created

APPENDIX C

Risk register and heat map

Eleven risks worth planning for on this host profile, positioned by likelihood and impact. Every one of them already has a control somewhere in the directive — the register simply says where.

FIGURE C.1 — RISK HEAT MAP: LIKELIHOOD AGAINST IMPACT



Position is judgement, not measurement — it is a prioritisation aid. What matters is that every marked cell has a named control in the table below, and that the two red-band risks (R-01, R-02) are checked *before* the run starts rather than discovered at minute forty.

TABLE C.1 — RISK REGISTER

ID	RISK	L × I	CONTROL IN	MITIGATION
R-01	MDM policy blocks kernel-driver installation	4 × 5 = 20	R6 · §4.1	Report as environmental blocker; capture MSI log; two attempts maximum.
R-02	Target volume has less than 45 GB free	5 × 3 = 15	G4 · §2.1	Offer alternate volume + machinefolder redirect, or free space first.
R-03	Unattended install stalls on the username prompt	3 × 4 = 12	§6.2	Preseed lowercase, rename in late_command, verify with id Kali.
R-04	VBS / Hyper-V owns the CPU — degraded execution	4 × 2 = 8	G5 · §2.1	Quantify the cost, present the trade-off, never disable VBS unasked.
R-05	ISO checksum mismatch — corrupt or tampered image	1 × 5 = 5	R5 · §5.3	Delete, retry once from the official mirror, halt on second failure.
R-06	Host thrashes under guest + browser memory pressure	3 × 3 = 9	§5.2 · §9	Drop guest to 2048 MB, core metapackage only, close MCP servers.
R-07	Protected artifact deleted during teardown	2 × 5 = 10	R4 · §3.2	Protected classes listed separately; second independent confirmation.
R-08	Run interrupted mid-install (sleep, MDM restart)	3 × 2 = 6	R2 · App. D	Resume from state.json; never re-run the destructive phase.
R-09	Hardcoded ISO filename 404s after a quarterly release	2 × 2 = 4	R5 · §5.3	Resolve the name from the /current/ listing at run time.

ID	RISK	L × I	CONTROL IN	MITIGATION
R-10	Guest Additions fail to build — kernel headers missing	2×3=6	§9	Install linux-headers-\$(uname -r) in-guest, rerun the installer.
R-11	Desktop icon invisible under OneDrive Known Folder Move	2×1=2	§8.3	Resolve the path with GetFolderPath('Desktop'); refresh explorer.

C.2 — Residual risk and the two pre-run checks

With every control in this directive applied, two risks remain materially unmitigated, and both are environmental rather than technical:

- ◆ **R-01 — MDM blocks the hypervisor driver.** No control in this document can fix it, by design: rule R6 forbids the only workarounds. If it fires, the build cannot complete on this host and the correct outcome is a clear blocker report naming the policy, not a partially built machine.
- ◆ **R-02 — insufficient disk.** Mitigable only with the operator's cooperation — freeing space or nominating another volume. The agent can measure and offer; it cannot decide what to delete.

Both are cheap to test *before* committing to a two-hour run, and both are checked in Phase 1 for exactly that reason. The residual risk after those checks is low enough that a run which clears Phase 1 has a high probability of reaching handover.

TWO-MINUTE PRE-RUN CHECK

```

PWSH run before ~10
      authorising s

# R-01 — can this host load a new
kernel driver?
(Get-CimInstance Win32_DeviceGuard
-Namespace `

root\Microsoft\Windows\DeviceGuard)
.CodeIntegrityPolicyEnforcementStat
us

# R-02 — is there anywhere to put
45 GB?
Get-Volume | Where-Object DriveType
-eq 'Fixed' |
    Select-Object DriveLetter,
    @{n='FreeGB';e=
    {[math]::Round($_.SizeRemaining/1GB
,1)}} |
    Sort-Object FreeGB -Descending

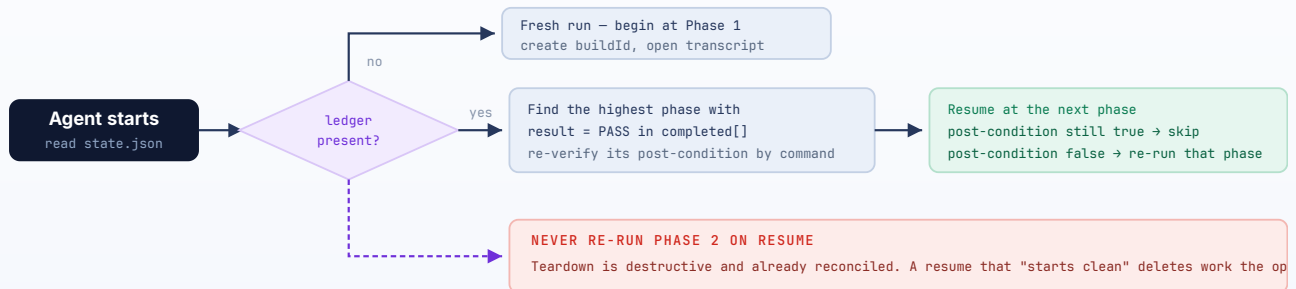
```

APPENDIX D

Resumable state machine and telemetry

Rule R2 promises that an interrupted run resumes rather than restarts. This appendix is the mechanism behind that promise: one ledger file, one resume rule, one telemetry schema.

FIGURE D.1 — RESUME LOGIC: WHERE A RE-INVOKED AGENT PICKS THE RUN BACK UP



Trusting the ledger alone is not enough — a phase marked PASS is still re-verified by its own post-condition command before it is skipped. The ledger says where to look; the host says what is true.

Phase post-conditions used on resume

PHASE	RE-VERIFICATION
1	<code>preflight.json</code> exists and is < 24 h old
2	<code>list vms</code> and <code>list hdds</code> both empty
3	<code>VBoxManage --version</code> returns 7.x
4	ISO present <i>and</i> re-hashed to the recorded SHA256
5	VM registered and guest reports <code>Linux</code>
6	<code>snapshot list</code> contains <code>clean-baseline</code>
7	<code>Test-Path</code> on the desktop <code>.lnk</code>

JSON file rewritten after every phase
C:\ProgramData\KaliVMBuild\state.json

```

{
  0: 29,
  1: 2,
  2: 30,
  3: { 4: 31, 5: 7.73 },
  6: 32,
  7: [
    { 8: 33, 9: 34,
      10: 35, 11: 36 }
  ],
  12: [
    { 13: 37, 14: 38, 15: [39],
      16: [], 17: 40, 18: 41 },
    { 19: 42, 20: 43, 21: [44],
      22: [45], 23: 46,
      24: 47 }
  ],
  25: { 26: 48, 27: 49, 28: 50 }
}
  
```

Telemetry emitted per phase (rule R7)

FIELD	TYPE	NOTES
id	string	<code>P1</code> ... <code>P7</code> . Stable across runs — this is the join key between the ledger, the transcript and the report.
result	enum	<code>PASS</code> <code>FAIL</code> <code>SKIPPED</code> . <code>SKIPPED</code> only ever appears on a resume.

FIELD	TYPE	NOTES
created	string[]	Paths, relative to <code>\$Root</code> where possible. An empty array is a claim, so make it true.
destroyed	string[]	Absolute paths. Only Phase 2 ever populates this.
elapsed	hh:mm:ss	Wall time for the phase, excluding time spent waiting on an operator decision.
at	ISO 8601	Completion timestamp, UTC, matching the transcript's clock.

APPENDIX E

Glossary

Terms used with a specific meaning in this directive.

Acceptance criterion	A statement about the finished system that is proved by a command, not asserted. A1–A9.	Metapackage	A Kali package that pulls in a curated tool set. <code>kali-linux-core</code> is minimal; <code>kali-linux-default</code> is the full desktop toolkit.
BITS	Background Intelligent Transfer Service — the Windows transfer engine used for the ISO because it resumes after a network drop.	NAT	Network Address Translation. The guest can reach out; nothing can reach in. The basis of the security posture in §7.4.
Gate	A pre-condition evaluated before an action. On failure the agent halts and reports. G1–G7.	Preseed	A Debian -installer answer file. Every question the installer would ask must have an answer, or the "unattended" install stops and waits.
Guest Additions	VirtualBox drivers and services installed <i>inside</i> the guest. They provide the guest properties the host reads for A6.	PUEL	Oracle's Personal Use and Evaluation Licence, which governs the Extension Pack. Free for personal use; commercial use requires a licence (§4.2).
HVCI	Hypervisor-protected Code Integrity, a.k.a. Memory Integrity. Part of VBS; competes with VirtualBox for CPU virtualization.	SLAT	Second Level Address Translation (EPT / RVI). Hardware page-table virtualization; without it a modern guest is unusably slow (G2).
Idempotent	Safe to run again. A phase that creates something must first check whether it already exists and reconcile.	Standing rule	A constraint in force for the whole run, not a step. R1–R7.
Ledger	<code>state.json</code> . The per-phase record that makes a run resumable (Appendix D).	VBS	Virtualization - Based Security. Windows runs its own hypervisor; VirtualBox then executes in a slower hosted mode (G5).
MDM	Mobile Device Management. Policy applied from Entra; can revoke local changes and block driver installation (R6).	VDI	VirtualBox Disk Image. Created dynamic here: 60 GB maximum, ~18 GB actually consumed.

APPENDIX F · TEAR-OUT

Operator quick-start card

Everything the person who owns this laptop needs, on one page, after the agent has gone.

KALI LINUX VIRTUAL MACHINE

LOGIN

Your machine, in five facts

Kali / bot

1 · STARTING IT

Double-click **Kali VM Control Center** on your desktop, then **Start (Window)**. The first boot takes about a minute.

2 · LOGGING IN

The username is **Kali** with a capital K. Linux logins are case-sensitive — **kali** will not work. The password is **bot**.

3 · SHUTTING IT DOWN

Use **Shut Down (ACPI)** in the panel, or shut down from inside Kali. Both are clean. Avoid closing the VM window with "Power off the machine".

4 · WHEN SOMETHING BREAKS

Click **Restore Baseline**. It rolls the machine back to its first-boot state — a clean install with nothing added. **Everything you saved inside the VM after that point is discarded**, so keep work you care about outside the guest or take your own snapshot first.

5 · THE ONE SAFETY RULE

IMPORTANT

The password is deliberately short. It is safe only because the VM has **no way in from the network** — it can reach out, nothing can reach it. If you ever switch it to *bridged* networking or set up port forwarding, **change the password first**.

MEMORY

The VM takes 3 GB of your 7.7 GB. Close browser windows before starting it, or the laptop will feel slow.

WHERE THINGS LIVE

Build logs and the control panel script: **C:\ProgramData\KaliVMBuild**. The disk image sits with the VM.

ADVANCED SETTINGS

VirtualBox Manager in the panel opens the full application if you need to change hardware settings.

QUICK TROUBLESHOOTING

"Login incorrect" every time	Capital K in Kali . Check Caps Lock is off.
Panel says unavailable	VirtualBox is not installed or was updated. Reopen the panel; if it persists, reinstall VirtualBox.
VM window is black after starting	Give it 60 seconds. If it stays black, shut down and start again.
Laptop becomes very slow	The VM is using 3 GB. Shut the VM down, or close other applications before starting it.

GUEST NAME

kali

as registered in VirtualBox

SNAPSHOT

clean - baseline

your rollback point

NETWORK

NAT only

outbound, no inbound path